



Queensland University of Technology
Brisbane Australia

This may be the author's version of a work that was submitted/accepted for publication in the following source:

Conforti, Raffaele, La Rosa, Marcello, Fortino, Giancarlo, ter Hofstede, Arthur, Recker, Jan, & Adams, Michael

(2013)

Real-time risk monitoring in business processes: A sensor-based approach.

Journal of Systems and Software, 86(11), pp. 2939-2965.

This file was downloaded from: <https://eprints.qut.edu.au/219676/>

© Consult author(s) regarding copyright matters

This work is covered by copyright. Unless the document is being made available under a Creative Commons Licence, you must assume that re-use is limited to personal use and that permission from the copyright owner must be obtained for all other uses. If the document is available under a Creative Commons License (or other specified license) then refer to the Licence for details of permitted re-use. It is a condition of access that users recognise and abide by the legal requirements associated with these rights. If you believe that this work infringes copyright please provide details by email to qut.copyright@qut.edu.au

License: Creative Commons: Attribution-Noncommercial-No Derivative Works 2.5

Notice: *Please note that this document may not be the Version of Record (i.e. published version) of the work. Author manuscript versions (as Submitted for peer review or as Accepted for publication after peer review) can be identified by an absence of publisher branding and/or typeset appearance. If there is any doubt, please refer to the published source.*

<https://doi.org/10.1016/j.jss.2013.07.024>

Real-Time Risk Monitoring in Business Processes: A Sensor-based Approach

Raffaele Conforti^a, Marcello La Rosa^{a,b}, Giancarlo Fortino^c, Arthur H. M. ter Hofstede^{a,d},
Jan Recker^a, Michael Adams^a

^a*Queensland University of Technology, GPO Box 2434, Brisbane QLD 4001, Australia*
E-mail: {raffaele.conforti,m.larosa,a.terhofstede,j.recker,mj.adams}@qut.edu.au

^b*NICTA Queensland Lab, Brisbane, Australia*

^c*Università della Calabria, Via P. Bucci, cubo 41C, 87036 Rende (CS), ITALY*
E-mail: g.fortino@unical.it

^d*Technische Universiteit Eindhoven, P.O. Box 513, 5600 MB Eindhoven, The Netherlands*

Abstract

This article proposes an approach for real-time monitoring of risks in executable business process models. The approach considers risks in all phases of the business process management lifecycle, from process design, where risks are defined on top of process models, through to process diagnosis, where risks are detected during process execution. The approach has been realized via a distributed, sensor-based architecture. At design-time, sensors are defined to specify risk conditions which when fulfilled, are a likely indicator of negative process states (*faults*) to eventuate. Both historical and current process execution data can be used to compose such conditions. At run-time, each sensor independently notifies a sensor manager when a risk is detected. In turn, the sensor manager interacts with the monitoring component of a business process management system to prompt the results to process administrators who may take remedial actions. The proposed architecture has been implemented on top of the YAWL system, and evaluated through performance measurements and usability tests with students. The results show that risk conditions can be computed efficiently and that the approach is perceived as useful by the participants in the tests.

1. Introduction

Business processes are constantly exposed to a wide range of risks. As demonstrated by the recent incidents in the finance sector (the €4.9B fraud at Société Générale [47]; the US\$2.3B UBS rogue trading scandal [33]), in the health sector (the Patel Inquiry [96, 44]) and in the aviation industry (terrorist attacks [101]), failures of process-driven risk management can result in sub-

stantial financial and reputational consequences, potentially threatening an organization's existence.

According to the AS/NZS ISO 31000 standard, a *business process risk* is the chance of something happening that will have a negative impact on the process objectives, and is measured in terms of likelihood and consequence [89]. Legislative initiatives

such as the Sarbanes-Oxley Act¹ and Basel II [8] in the finance sector have highlighted the pressing need to better manage business process risks. As a consequence of these mandates, organizations are now seeking new ways to *control* process-related risk and are attempting to incorporate it as a distinct view in their operational management [100, 75]. However, whilst conceptually appealing, to date there is little guidance as to how this can be done concretely. Currently, the disciplines of process management and risk management are largely disjoint and operate independently of one another [92]. In industry they are usually handled by different organizational units, where the disconnectedness of the risk control systems in place provide an environment for potentially massive risk exposure.

For example, in the UBS case, while the bank employed best-practice financial risk management strategies, the fraud enacted by a single trader exploiting a confirmation activity in the exchange-traded funds process may potentially have continued for years without being detected [66]. UBS later admitted that the disengaged nature of their risk and operational systems meant that, while some unexplained activity had been detected, it was not sufficiently investigated, nor were existing risk controls enforced [32, 33]. Rather, regular risk reports, internal audits and external reviews repeatedly delivered positive outcomes, leading to management complacency and a lack of healthy mistrust [90]. Another case in point is that of KfW Bankengruppe, which in 2008 transferred €300M to Lehmann Brothers using an automated transfer facility hours before Lehmann declared bankruptcy [79]. Such examples il-

lustrate a clear misalignment between risk management and the (automated) enactment of business operations. Disasters like these are likely to become more prevalent as businesses increasingly rely on heterogeneous, complex and distributed ICT systems for their day-to-day operations. Of course, proper risk management is not only pivotal in the finance sector, but also plays a central role in many other domains, including health (e.g., limiting the spread of diseases and minimizing exposure to litigation), mining (e.g., physical safety), and aviation (e.g., maintenance and terrorism).

Within academia, recent research has centered on the characterization of process-related risks. While risk metrics and mitigation procedures have been broadly explored [89], such generalized strategies fail to target the specific requirements of process-centric operationalization. Conversely, numerous techniques have been developed for specific domains [3, 57, 43, 13], but the nature of such techniques renders them unable to offer strategies that can readily be applied more generally.

Effective risk management is imperative for organizational survival, and incidents such as those described above reveal that proper integration of risk management with process management is now vital. Further, a focus on risk analysis alone is no longer adequate. Rather, active, real-time risk detection, analysis and mitigation is required. Real-time risk detection is particularly essential in mission-critical business processes or those involving life-saving activities [20, 65]. Through the integration of the risk management and process management domains, a number of key benefits may be realized, from incorporating risk analysis and mitigation strategies during the process design phase, to monitoring

¹www.gpo.gov/fdsys/pkg/PLAW-107pub1204

and mitigating emerging risks during process execution, to post-execution risk analysis using process log data.

We propose a concrete approach for real-time monitoring of risks in executable business process models. This is achieved by incorporating risk management into all phases of the BPM lifecycle: from process design, where high-level risks defined via a risk analysis method are mapped down to specific process model elements such as activities, resources and data, through to process diagnosis, where risks are detected during process execution. By automating risk detection, the interested users (e.g. a process administrator or a risk manager) can be notified as early as a risk is detected (i.e. in real-time), such that remedial actions can be taken to rectify the current process instance, and prevent an undesired state of the process (*fault* for short), from occurring. Based on historical data, we can also compute the probability of a risk at run-time, and compare it to a threshold, so as to notify the user only when the risk is no longer tolerable.

The proposed approach is operationalized via a distributed, sensor-based architecture on top of Business Process Management Systems (BPMSs). Each sensor is coupled with a risk condition capturing the situation upon which the risk of a given fault may occur. Sensors are defined at design-time on the executable process model. Conditions can be determined via a query language that can fetch both historical and current execution data from the logs of the BPMS. At run-time sensors are registered with a central sensor manager. At a given sampling rate, or based on the occurrence of a specific event, the sensor manager retrieves and filters all data relevant for the various sensors (as it is logged by the BPMS en-

gine), and distributes it to the relevant sensors. If a sensor condition holds, i.e. if the probability of the associated risk is above a given threshold, the sensor alerts the sensor manager which in turn notifies the monitoring component of the BPMS. The distributed nature of the architecture guarantees that there is no performance overhead on the BPMS engine, and thus on the execution of the various process instances. We implemented this architecture on top of the YAWL system [95]. We extended the YAWL Editor to cater for the design of risk sensors, and equipped the run-time environment with a sensor manager service that interacts with YAWL's monitoring service and execution engine. Finally, to facilitate the definition of process risks, we implemented a set of risk templates for various categories, such as organizational, data-related and technology-related. Such templates are abstract risk definitions which users need to bind to concrete process elements.

To prove the feasibility of the proposed approach, we used fault tree analysis [21] (a well-established risk analysis method) to identify risk conditions in a reference process model for logistics, in collaboration with an Australian risk consultant. These risks embrace different process aspects such as tasks' order dependencies, involved resources and business data, and relate to historical data where needed, to compute risk probabilities. We expressed these conditions via sensors in the YAWL system, and measured the time needed to compute these conditions at run-time. The tests showed that the sensor conditions can be computed in a matter of milliseconds without impacting on the performance of the running process instances. Finally, we conducted a usability analysis of the approach and its im-

plementation, by inviting 21 students to report on their experiences while using the approach for analyzing and defining risk sensor conditions. The results from this experiment show that the approach is being perceived as interesting and useful, especially by novice users, confirming an intention of the participants to use it.

This paper is organized as follows. Section 2 illustrates the running example in the logistics domain. Section 3 describes our risk-aware BPM approach while Section 4 presents the sensor-based architecture to implement this approach. Section 5 formalizes the abstract syntax of the language proposed for risk detection while Section 6 shows how the risks defined on the running example can be modeled via this language. Next, Section 7 proposes a set of risk templates to facilitate the definition of process risks. Section 8 illustrates the implementation of the sensor-based architecture, while Section 9 reports on the results of the performance and usability evaluations. Section 10 covers related work while Section 11 concludes the paper. Appendix A provides the complete abstract syntax of the language for sensor conditions; Appendix B provides a short description of the actions defined in the language; Appendix C describes the nested loops that can be executed during the verification of a sensor condition while Appendix D describes the functions that can be invoked on variables (that are resources) during the verification of a sensor condition. Finally, Appendix E provides the questionnaire used in the usability tests.

2. Running Example

In this section we use an example to illustrate how the risk of possible faults to occur

during a business process execution can be detected as early as possible. In particular, we show how risks can be expressed in terms of process-specific aspects such as tasks occurrence, data or available resources. Figure 1 describes the payment subprocess of an order fulfillment business process which is inspired by the VICS industry standard for logistics [98]. The notation used to represent this example is that of YAWL [95], although a deep knowledge of this language is not required.

This process starts after the freight has been picked up by a carrier and deals with the shipment payment. The first task is the production of a Shipment Invoice containing the shipment costs related to a specific order for a specific customer. If shipments have been paid in advance, all that is required is for a Finance Officer to issue a Shipment Remittance Advice specifying the amount being debited to the customer. Otherwise, the Finance Officer issues a Shipment Payment Order that needs to be approved by a Senior Finance Officer (who is the superior of this Finance Officer). At this point, a number of updates may be made to the Shipment Payment Order by the Finance Officer that issued it, but each of these needs to be approved by the Senior Finance Officer. After the document is finalized and the customer has paid, an Account Manager can process the shipment payment by specifying the balance. If the customer underpaid, the Account Manager needs to issue a Debit Adjustment, the customer needs to pay the balance and the payment needs to be reprocessed. A customer may also overpay. In this case the Account Manager needs to issue a Credit Adjustment. In the latter case and in case of a correct payment, the shipment payment process is completed.

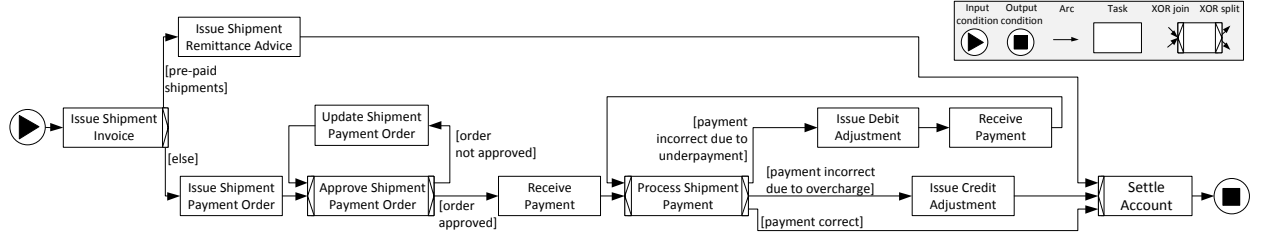


Figure 1: Order-Fulfillment: Payment subprocess.

In collaboration with a risk analyst of an Australian consulting company, we identified four faults that can occur during the execution of this payment subprocess. In order to prevent the occurrence of these faults, for each of them we also defined an associated risk condition by using fault tree analysis [21]. Accordingly, each risk condition is expressed as a set of lower-level boolean events which are organized in a tree via logical connectives such as ORs, ANDs and XORs, an example of fault tree is shown in Figure 2.

The first fault is an *overtime process* fault. A Service Level Agreement (SLA) for a process or for a given task within a process, may establish that the process (or task) may not last longer than a Maximum Cycle Time MCT , otherwise the organization running the process may incur a pecuniary penalty. In our case, an overtime fault occurs if an instance of the payment subprocess is not completed within an MCT of five days.

To detect the risk of overtime fault at run-time, we should check the likelihood that the running instance does not exceed the MCT based on the amount of time T_c expired at the current stage of the execution. Let us consider T_e as the remaining cycle time, i.e. the amount of time estimated to complete the current instance given T_c . Then the probability of exceeding MCT

can be computed as $1 - MCT/(T_e + T_c)$ if $T_e + T_c > MCT$ and is equal to 0 if $T_e + T_c \leq MCT$. If this probability is greater than a tolerance value (e.g. 60%), we notify the risk to the user. The estimation of the remaining cycle time is based on past executions of the same process and can be computed using the approach in [1], whereby using an annotated transition system an estimation of the remaining cycle is calculated by replaying the trace on the transition system and returning the expected time annotated on the last node visited (see Section 9 for more details).

The second fault is related to the resources participating in the process. The Senior Finance Officer who has approved a Shipment Payment Order for a given customer, must have not approved another order by the same customer in the last d days, otherwise there is an *approval fraud*. This fault is thus generated by the violation of a four-eye principle across different instances of the Payment subprocess.

To detect the risk of this fault we first have to check that there is an order, say order o of customer c , to be approved. This means checking that an instance of task Approve Shipment Payment Order is being executed. Moreover, we need to check that either of the following conditions holds: i) o has been allocated to a Senior Finance Officer who has already approved another

order for the same customer in the last d days; or ii) at least one Senior Finance Officer is available who approved an order for customer c in the last d days and all other Senior Finance Officers who never approved an order for c during the last d days are busy. The corresponding fault tree is shown in Figure 2.

The third fault relates to a situation where a process instance executes a given task too many times. This situation typically occurs in the context of loops. Not only could this lead to a process slowdown but also to a “livelock” if the task is in a loop whose exit condition is purposefully never met. In general, given a task t a maximum number of allowable executions of t per process instance $MAE^i(t)$ can be fixed as part of the SLA for t . With reference to the Payment subprocess, this can occur for example if task Update Shipment Payment Order is re-executed five times within the same process instance. We call this an *order unfulfillment* fault.

To detect the risk of this fault at run-time, we need to check if: i) an order o is been updated (i.e. task Update Shipment Payment Order is currently being performed for order o); and ii) it is likely that this order will be updated again (i.e. task Update Shipment Payment Order will be repeated within the same process instance). The probability that the number of times a task will be repeated within the same instance of the Payment subprocess is computed by dividing the number of instances where the MAE^i for task Update Shipment Payment Order has been reached, over the number of instances that have executed this task at least as many times as it has been executed by the current instance, and have completed. The tolerance value indicates a threshold above which the risk should be

notified to the user. For example, if this threshold is 60% for task t , a risk should be raised if the probability of $MAE^i(t)$ is greater than 0.6.

The fourth fault is an *underpayment fraud*. It relates to a situation in which a given task is executed too many times across multiple process instances. Similar to the previous fault, given a task t we can define a maximum number of allowable executions of t per process $MAE^p(t)$ as part of the SLA for p . In our example, this type of fault occurs when a customer underpays more than three times within the last five days.

To detect the risk of underpayment fraud, we need to check if: i) a debit adjustment is currently being issued to a customer c (i.e. task Issue Debit Adjustment is currently being performed for customer c); and ii) it is likely that the maximum number of debit adjustments will be issued to the same customer in a d -day time frame. The probability that MAE^p is reached for task Issue Debit Adjustment of customer c in d days is computed by dividing the number of customers for which the MAE^p for task Issue Debit Adjustment has been reached within d days, over the number of customers for which this task has been executed at least as many times as it has been executed for c within d days. If this probability is above a tolerance value, the risk should be raised and the user notified. Similar to the previous risk, the tolerance value indicates a threshold above which this risk should be notified to the user. The corresponding fault-tree is shown in Figure 2.

3. Risk-aware Business Process Management

As we have seen in the context of the payment example, a *process-related fault* is

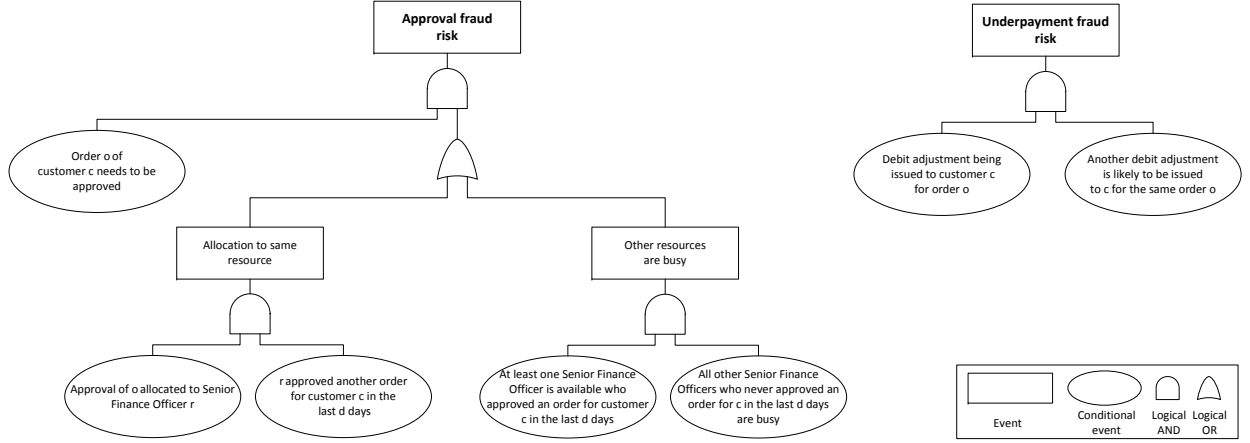


Figure 2: The fault trees for Approval Fraud and Underpayment Fraud.

an unwanted situation that will negatively impact upon the objectives of the business process (e.g. the violation of a policy may lead to a process instance being interrupted), and is identified by a *fault condition* and a *consequence*. Identifying a fault in a process requires determining the condition upon which the fault occurs. For example, in the payment subprocess, we have an underpayment fraud if a customer underpays more than three times within a five-day time frame.

However, a *fault condition* holds only when the associated fault has occurred, which is typically too late to avoid a process failure. Indeed, we need to be able to estimate the risk of a process fault, i.e. if, and possibly with what likelihood, the fault will occur in the future. A *process-related risk* is the chance of a process-related fault happening and is measured in terms of consequence (inherited from it related fault) and likelihood. Early risk detection allows process users to promptly react with countermeasures, if any, to prevent the related fault from occurring at all.

We use the notion of *risk condition*, as opposed to fault condition, to describe the

set of events that lead to the possibility of a fault to occur in the future. In order to evaluate risk conditions “on-line”, i.e. while a process instance is being executed, we need to consider the current state of the BPMS. This means knowing the state of all running instances of any process (and not only the state of the instance for which we are computing the risk condition), the resources that are busy and those that are available, and the values of the data variables being created and consumed. Moreover, we need to know the historical data, i.e. the execution data of all instances that have been completed. In particular, we can use historical data to estimate the probability of a given fault to occur, i.e. the *risk probability*. For example, for the underpayment fraud, we can estimate the likelihood that another debit adjustment is being issued for a given combination of customer/order (historical data), given that one such debit adjustment has just been issued (current data). To obtain a boolean risk condition, we compare the risk probability that we obtain with a tolerance value, such that the condition holds if the risk probability exceeds the given threshold. For example, we

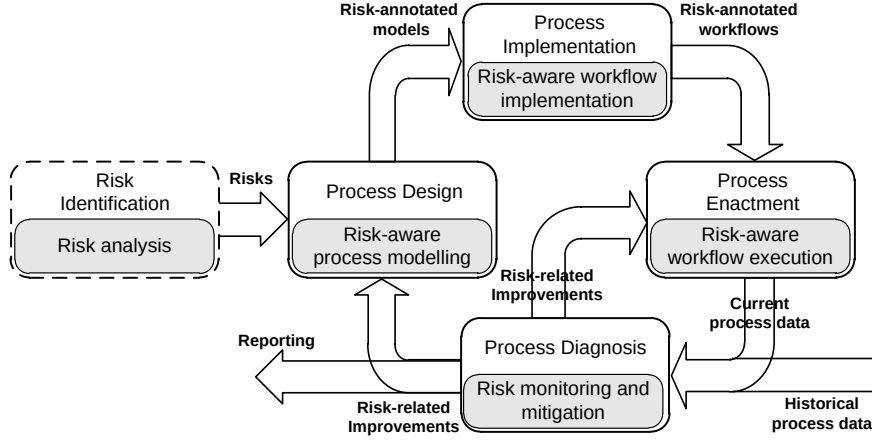


Figure 3: Risk-aware Business Process Management lifecycle.

raise the risk of underpayment fraud if the risk probability is greater than 60%.

In other cases, we may avoid to embed a risk probability in the risk condition, if we are able to determine the occurrence of a set of events which directly leads to a high risk. This is the case of the approval fraud, where both the events “Allocation to same resource” and “Other resources are busy” already signal a high risk of approval fraud.

Based on these considerations, we present a novel approach for on-line risk detection in business processes. The focal idea of this approach, shown in Figure 3, is to embed elements of risk into all four phases of the traditional BPM lifecycle [31].

Input to this “risk-aware” BPM lifecycle is a *Risk Identification* phase, where risk analysis is carried out to identify risks in the process model to be designed. Traditional risk analysis methods such as FTA (as seen in the previous section), Root Cause Analysis [50] or CORAS [58], can be employed in this phase. The output of this phase is a set of risks, each expressed as a risk condition. It is important to notice that not all risks identified during risk analysis can be avoided by preemptively modifying the

business process in which they may eventually occur. For example, an overtime risk, i.e. the risk of not completing a process instance within the required time limit, depends on the particular way in which the given process instance is being executed. Our approach specifically targets those risks that can only be detected during process execution.

Next, in the *Process Design* phase, these high-level risk conditions are mapped down to process model-specific aspects. For example, the condition “debit adjustment being issued to customer *c* for order *o*” is mapped to the occurrence of a specific task, namely “Issue Debit Adjustment” in the Payment process model. The result of this second phase is a risk-annotated process model. In the next phase, *Process Implementation*, these conditions are linked to workflow-specific aspects, such as content of variables, and resource allocation states. For example, “customer *c*” is linked to the **Customer** element of the XML representation of the **Debit Adjustment** document. Process Implementation may be integrated with Process Design if the language used at design-time is executable (e.g. BPMN 2.0 or

YAWL).

The risk-annotated workflow model resulting from Process Implementation is then executed by a risk-aware process engine during *Process Enactment*. Historical data stored in process logs, and current execution data coming from process enactment, are filtered, aggregated and analyzed in the *Process Diagnosis* phase, in order to evaluate the various risk conditions. When a risk condition evaluates to true, the interested users (e.g. a process administrator) are notified and reports can also be produced during this phase for auditing purposes. Finally, this phase can trigger changes to the current process instance, in order to mitigate the likelihood of a fault to occur, or in the underlying process model, to prevent a given risk from occurring ever again.

In the next section we describe a sensor-based architecture to operationalize our approach, according to the enhanced BPM lifecycle described above. In particular, the approach does not cover Risk Identification, since the deliverables of this phase (e.g. a fault tree) are already available in any organization that conducts risk analysis. As for the last phase of the lifecycle (Process Diagnosis), in this paper we only focus on the risk monitoring part. Risk mitigation is not in the scope of this paper as we proposed a solution to this problem in separate work [24]. More information on this solution is provided in the Related Work section.

4. Sensor-based Realization

In order to realize our risk-aware BPM lifecycle, we devised an approach based on sensors. In a nutshell, the idea is to capture risk and fault conditions via sensors, and then monitor these sensors during process

execution. An overview of this approach is shown in Figure 4 using the BPMN 2.0 notation [68].

Sensors are defined during the *Process Design* and *Process Implementation* phases of our risk-aware BPM lifecycle (see Figure 3), for each process model for which the presence of risks and/or faults need to be monitored. If the process model is specified via an executable language, then these two phases coincide.

A sensor is defined through a boolean *sensor condition*, constructed on a set of process variables, and a *sensor activation trigger*. Process variables are used to retrieve information from the specific instance in which the sensor condition will be evaluated as well as from other instances, either completed or still running. For example, we can use variables to retrieve the resource allocated to a given task, the value of a task variable, or the status of a task. Process instances can either be identified based on the current instance (e.g. the last five instances that have been completed before the current one), or based on the fulfillment of a *case condition* (e.g. “all instances where a given resource has executed a given task”). The sensor condition can represent either a *risk condition* associated with a fault, or a *fault condition*, or both. If both conditions are specified, the fault condition is evaluated only if the risk condition evaluates to true. For example, the sensor will check if an overtime process fault has occurred in a process instance only if first the risk of such fault has first been detected, based on the estimation of the remaining cycle time for this instance. Finally, the sensor activation trigger can be either a timer periodically fired according to a sampling rate (e.g. every 5 minutes), or an event emitted by the process engine (e.g. the completion

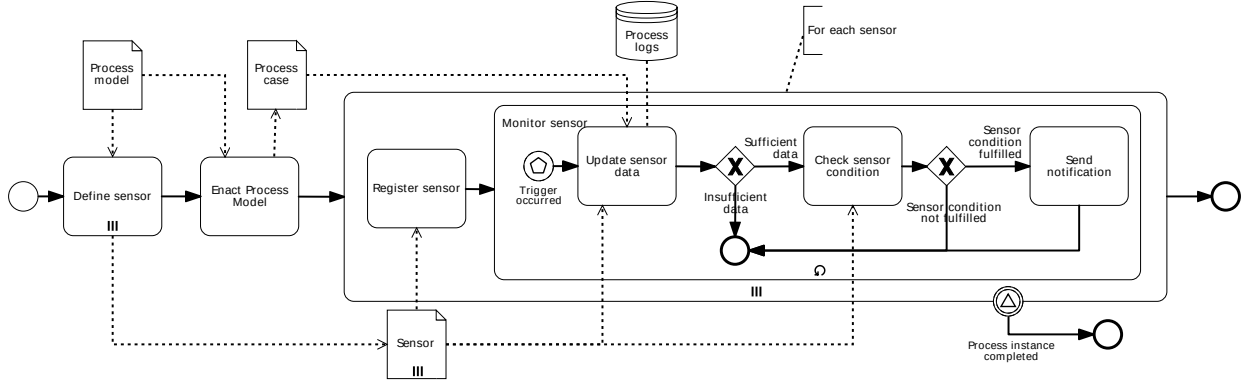


Figure 4: Realization of risk-aware BPM lifecycle via sensors.

of a task).

During *Process Enactment*, the defined sensors are registered with a *sensor manager*, which activates them. In the *Process Diagnosis* phase, which starts as soon as the process is enacted, the activated sensors receive updates on the variables of their sensor conditions according to their trigger (timer or event). When a sensor receives an update, it checks its sensor condition. If the condition holds, a notification is sent from the sensor to the monitor service of the BPMS.

The sensor manager relies on three interfaces to interact with the BPMS (see Figure 5(a)):

- *Engine interface*, used to register a sensor with a particular event raised by the BPMS engine. When the event occurs the sensor is notified by the sensor manager.
- *Database interface*, used to query the BPMS database in order to collect current and historical information.
- *Monitor interface*, used to notify the detection of risks and faults to the monitor service of the BPMS.

These interfaces can be implemented by the vendor or user of the BPMS where the sensor manager needs to be installed. In this way, our sensor manager can virtually be interfaced with any BPMS. As an example, the conceptual model of the database interface is showed in Figure 5(b), where methods have been omitted for space reasons. This conceptual model is inspired by the reference process meta-model of the WfMC [45], in order to cover as many aspects as possible of a workflow model, and meantime, to remain as generic as possible. For example, class *WorkflowDefinition* allows one to retrieve information about the process model where the sensor is defined, such as process identifier and name, while class *SubProcess* allows one to retrieve information about a specific subprocess, and so on. This interface should be implemented according to the characteristics of the specific database used in the BPMS at hand. For an efficient use of the interface, one should also define indexes on the attributes of the BPMS database that map the underlined attributes in Figure 5(b). These indexes have been determined based on the types of queries that can be defined in our sensor definition language.

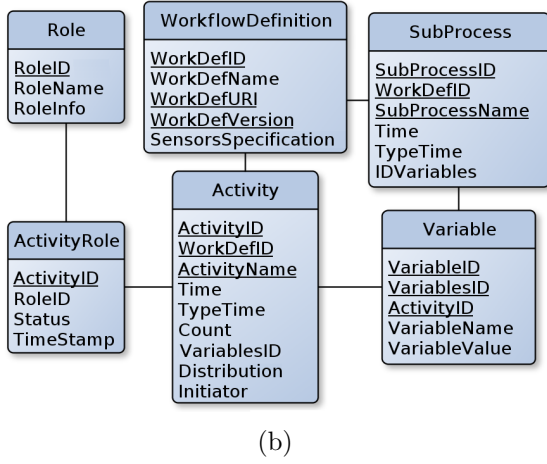
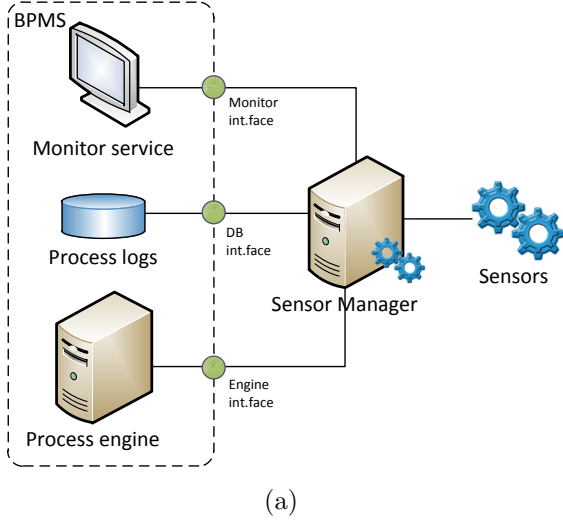


Figure 5: Sensor-based architecture (a); Database Interface schema model (b).

An alternative approach to achieve the portability of the sensor manager, would be to read the BPMS logs from a standard serialization format such as OpenXES [38]. However, as we will show in Section 9, this solution is rather inefficient.

The advantages of using sensors are twofold. First, their conditions can be monitored while the process model is being executed, i.e. in real-time. Second, according to a distributed architecture, each sensor takes care of checking its own condition af-

ter being activated by the sensor manager. In this way, potential execution slowdowns are avoided (e.g., the process engine and the sensor manager could be deployed onto two different machines).

5. Sensor Definition Language: Abstract Syntax

In this section we describe the abstract syntax [63] of the sensor definition language. A detailed description of all elements of this abstract syntax is reported in Appendix A. The sensor definition language shares common aspects with programming languages such as with query languages, since it is inspired to JAVA from a syntactic point of view and to SQL for its semantics. During the definition of the sensor definition language we took in consideration the design principles discussed in [39, 9], in particular: i) expressibility: is the language expressive?; ii) completeness: can the language describe all objects of interest?; iii) extensibility: can the language be easily extended?; iv) formal foundation: is the language formally defined in order to guarantee unambiguity and executability?.

The design principle of expressibility led to the introduction of various constructs such as conditional and looping ones (discussed later in this section). The principle of completeness led to the definition of elements of the language that allow the retrieval of all information related to a work item or a process instance available to the workflow engine. In order to make the language extensible, we structured it hierarchically allowing an easy introduction of new constructs. Finally, the formal foundation of the language drove the implementation of the language and of its interpreter.

The definition of a sensor requires a *risk*

condition, a boolean *fault condition*, a *sensor activation trigger*, and a *consequence* which represents the gravity of the impact the fault will have on the company in case it occurs. The trigger can be a timer based on a sampling rate, or an event produced by the engine interface. Risk condition and fault condition are constructed on a set of process variables. A variable is defined with a mapping among a *varName* and an *Info*.

$$\begin{aligned}
\text{Sensor} &\triangleq v : \text{Variables}; t : \text{Trigger}; \\
&\quad \text{riskCond} : \text{RiskCon}; \\
&\quad \text{faultCond} : \text{BoolCon}; \\
&\quad \text{consequence} : \text{MaCon}; \\
\text{Trigger} &\triangleq \text{timer} \mid \text{event} \\
\text{Variables} &\triangleq \text{Assignment}^+ \\
\text{Assignment} &\triangleq \text{result} : \text{varName}; i : \text{Info}
\end{aligned}$$

This *Info* can be a constant (using *Definition*), or the result of a function executed on a variable (using *VarFun*), or a piece of information collected from the process instance (using *CaseExp* or *CaseEleExp*). We use a *CaseExp* if the information is related to the process instance itself, while a *CaseEleExp* if the information is related to an element of a process instance (that must be specified using *TaskOrNet* that identifies a task by *taskLabel* or a net by *netName*).

$$\begin{aligned}
\text{Info} &\triangleq \text{Definition} \mid \text{VarFun} \mid \\
&\quad \text{CaseExp} \mid \text{CaseEleExp} \\
\text{VarFun} &\triangleq \text{ResCon} \mid \text{ResSimFun} \mid \\
&\quad \text{ResComFun} \\
\text{Definition} &\triangleq c : \text{constant} \\
\text{CaseEleExp} &\triangleq ce : \text{CaseExp}; ton : \text{TaskOrNet} \\
\text{TaskOrNet} &\triangleq \text{taskLabel} \mid \text{netName}
\end{aligned}$$

When we use a *CaseExp* we must specify the instances of interest (using *CaseIDStat*), and the action that identifies the piece of information (using *Action*). Such *Action* can either be an information related to a predicate function, a predicate

function with input, a task or net variable (i.e. a variable at the level of tasks or a variable at the level of processes), or a task or net subvariable (i.e. a subvariable of a task variable or a net variable, respectively). An instance can be identified in various ways, by its position among all the instances of the same process model (using *absExp*), by its position respect the current instance (using *relExp*), or by the fulfillment of some conditions (using *CaseConSet*).

$$\begin{aligned}
\text{CaseExp} &\triangleq cis : \text{CaseIDStat}; a : \text{Action} \\
\text{CaseIDStat} &\triangleq \text{absExp} \mid \text{relExp} \mid \text{CaseConSet} \\
\text{Action} &\triangleq \text{predAct} \mid \text{inputPredAct} \mid \\
&\quad \text{taskOrNetVar} \mid \text{SubVarExp} \\
\text{SubVarExp} &\triangleq \text{var}^+
\end{aligned}$$

These conditions can be on the identifier of the process instance (using *CaseParam*), or on a piece of information related to a task or a net (using *CaseCon*). It is also possible to specify multiple conditions that are obtained by the conjunction of several *CaseParam* and *CaseCon* elements (using *CaseConExp*).

$$\begin{aligned}
\text{CaseConSet} &\triangleq \text{CaseCon} \mid \text{CaseParam} \mid \\
&\quad \text{CaseConExp} \\
\text{CaseConExp} &\triangleq ccs_1, ccs_2 : \text{CaseConSet}; \\
&\quad bo : \text{BoolOp} \\
\text{CaseCon} &\triangleq ton : \text{TaskOrNet}; a : \text{Action}; \\
&\quad co : \text{CompOp}; rhe : \text{RHExp} \\
\text{CaseParam} &\triangleq i : \text{idFun}; co : \text{CompOp}; \\
&\quad rhe : \text{RHExp} \\
\text{CompOp} &\triangleq le \mid leq \mid ge \mid geq \mid eq \mid \\
&\quad \text{contains} \mid \text{isContained}
\end{aligned}$$

Whether using a *CaseParam* or a *CaseCon* it will be compared with a *RHExp*. A *RHExp* can be a *constant*, a *function*, a *varName*, or an expression containing those elements (using *RHExpSet*).

$$\begin{aligned}
\text{RHExp} &\triangleq \text{constant} \mid \text{function} \mid \\
&\quad \text{varName} \mid \text{RHExpSet} \\
\text{RHExpSet} &\triangleq rhe_1, rhe_2 : \text{RHExp}; o : \text{Op}
\end{aligned}$$

Once all the variables have been specified the risk condition can be defined. A *RiskCon* is composed of two *MaCon* elements, one is the risk likelihood and the other is the risk threshold. A risk condition evaluates true if the likelihood exceeds the threshold. A *MaCon* is an arithmetical expression that can use constants, variables, results of functions invoked on variables (using *ResSimFun*, and *ResComFun*), and the results obtained by the execution of loops (using *MaFor*, and *FixMaFor*). A *MaCon* may be in a conditional form, represented as *MaITE*, or be a normal expression represented as *MaExp*, despite it is always possible to have conditional elements inside a normal expression. A conditional expression *MaITE* is composed of a *BoolCon* representing the *if* and two *MaCons* representing the *then* and *else* clauses, respectively.

$$\begin{aligned}
RiskCon &\triangleq riskL, riskT : MaCon \\
MaCon &\triangleq MaITE \mid MaFor \mid ResSimFun \mid \\
&\quad MaExp \mid FixMaFor \mid constant \mid \\
&\quad ResComFun \mid varName \\
MaITE &\triangleq if : BoolCon; then, else : MaCon
\end{aligned}$$

From the definition of the *MaExp* element on, the syntax describes an arithmetical expression. In fact a *MaExp* element can be solved via the analysis of an unary expression *MaUnExp* (i.e. a negative expression) or a binary expression *MaBinExp*, that is an arithmetical operation between two *MaCons*.

$$\begin{aligned}
MaExp &\triangleq MaUnExp \mid MaBinExp \\
MaUnExp &\triangleq s : sub; me : MaCon \\
MaBinExp &\triangleq me_1, me_2 : MaCon; mo : MaOp \\
MaOp &\triangleq add \mid sub \mid mul \mid div \mid exp \mid mod
\end{aligned}$$

As said before a condition expression is defined using a *BoolCon*. A *BoolCon* is a boolean expression that can use constants, variables, results of functions invoked on variables (using *ResCon*), and the results obtained by the execution of loops (using

BoolFor, and *FixBoolFor*). A *BoolCon* may be in a conditional form, represented as *BoolITE*, or be a normal expression represented as *BoolExp*, despite it is always possible to have conditional elements inside a normal expression. A conditional expression *BoolITE* is composed of three *BoolCons* representing the *if*, the *then*, and the *else*.

$$\begin{aligned}
BoolCon &\triangleq BoolITE \mid BoolExp \mid BoolFor \mid \\
&\quad FixBoolFor \mid Comp \mid ResCon \mid \\
&\quad varName \mid constant \\
BoolITE &\triangleq if, then, else : BoolCon
\end{aligned}$$

The *BoolExp* element represents a boolean expression that is describes from this point on. In fact a *BoolExp* element can be solved via the analysis of an unary expression *BoolUnExp* or a binary expression *BoolBinExp*, that can contain the result of a comparison *Comp*, and so on.

$$\begin{aligned}
BoolExp &\triangleq BoolUnExp \mid BoolBinExp \\
BoolUnExp &\triangleq n : neg; e : BoolCon \\
BoolBinExp &\triangleq e_1, e_2 : BoolCon; bo : BoolOp \\
BoolOp &\triangleq and \mid or \\
Comp &\triangleq ce_1, ce_2 : CompElem; \\
&\quad co : CompOp \\
CompElem &\triangleq MaCon \mid ResListFun \mid \\
&\quad varName \mid constant \\
CompOp &\triangleq l \mid leq \mid eq \mid geq \mid g \mid noteq
\end{aligned}$$

The functions that can be invoked on a variable are identified by the elements: *ResCon*, *ResListFun*, *ResSimFun*, and *ResComFun*. These elements can be used only in specific points of the syntax since the result returned can be a boolean, or a

list, or a number.

$$\begin{aligned}
ResCon &\triangleq res : varName; a : Activity \\
ResListFun &\triangleq result : varName; \\
&\quad lrf : listResFun \\
ResSimFun &\triangleq resource : varName; \\
&\quad srf : simResFun \\
ResComFun &\triangleq res_1, res_2 : varName; \\
&\quad crf : comResFun \\
Activity &\triangleq resource : varName; \\
&\quad lrf : ResListFun
\end{aligned}$$

The constructs *MaFor* and *BoolFor* are used to execute nested loops. The definition of one of these elements requires to specify the type of loop (*TypeBF* for *BoolFor* or *TypeMF* for *MaFor*), the list of variables (i.e. *ListResult*) that will be used to create the nested loops (one loops for each variable), and the expression that must be executed. The constructs *FixMaFor* and *FixBoolFor* are nested loops like *MaFor* and *BoolFor* for which the variable *fixEle* determines the number of loops that need to be executed. The *TypeBF* and *TypeMF* describe how the results of each execution will be joined. It is possible to choose among four types of loops: and (i.e. logic AND), or (i.e. logic OR), add (i.e. addition), and mul (i.e. multiplication).

$$\begin{aligned}
MaFor &\triangleq t : TypeMF; lr : ListResult; \\
&\quad fme : ForMaCon \\
FixMaFor &\triangleq fixEle : varName; mf : MaFor \\
TypeMF &\triangleq add \mid mul \\
BoolFor &\triangleq t : TypeBF; lr : ListResult; \\
&\quad fbe : ForBoolCon \\
FixBoolFor &\triangleq fixEle : varName; bf : BoolFor \\
TypeBF &\triangleq and \mid or \\
ListResult &\triangleq varName^+
\end{aligned}$$

As for a condition also for loops it is possible to assume conditional or normal forms. The expression executed inside a loop is similar to the condition expression but with some variations. The *ForMaCon* is a arithmetical expression that can use constants

and variables, the difference with an *MaCon* is that it is not possible to use loops or functions inside this type of expression.

$$\begin{aligned}
ForMaCon &\triangleq ForMaITE \mid ForMaExp \mid \\
&\quad constant \mid varName \\
ForMaITE &\triangleq if : ForBoolCon; \\
&\quad then, else : ForMaCon \\
ForMaExp &\triangleq ForMaUnExp \mid \\
&\quad ForMaBinExp \\
ForMaUnExp &\triangleq s : sub; me : ForMaCon \\
ForMaBinExp &\triangleq me_1, me_2 : ForMaCon; \\
&\quad mo : MaOp
\end{aligned}$$

The *ForBoolCon* is a boolean expression and does not allow the use of loops or functions as the *ForMaExp*. Clarified this two points, all the elements the name of which starts with *For* are equals to the elements used by an *BoolCon*.

$$\begin{aligned}
ForBoolCon &\triangleq ForBoolITE \mid ForBoolExp \mid \\
&\quad ForComp \mid varName \mid \\
&\quad varName \mid constant \\
ForBoolITE &\triangleq if, then, else : ForBoolExp \\
ForBoolExp &\triangleq ForBoolUnExp \mid \\
&\quad ForBoolBinExp \\
ForBoolUnExp &\triangleq n : neg; e : ForBoolCon \\
ForBoolBinExp &\triangleq e_1, e_2 : ForBoolCon; \\
&\quad bo : BoolOp \\
ForComp &\triangleq ce_1, ce_2 : ForCompElem; \\
&\quad co : CompOp \\
ForCompElem &\triangleq ForMaCon \mid varName \mid \\
&\quad constant
\end{aligned}$$

6. Risk Definition for the Running Example

The abstract syntax defined in Section 5 has been used as a formal foundation for the definition of the concrete syntax of our language for sensor conditions. During the realization of the concrete syntax we followed the dot notation typical of object-oriented languages, whereby a “.” is used

as a separator between the object on which a function is requested and the function itself. In particular, whenever a function “a()” needs to be invoked on an object “b”, the structure used is “b.a()”. In the context of our language we follow this hierarchy: “case.task.action()”, “case.net.action()”, or “case.action()” for the definition of variables, and “task.action()”, or “net.action()” for the identification of cases which satisfy certain conditions. Finally, another aspect in common with object-oriented languages is the declaration of variables. In our concrete syntax, conditions can only be defined using variables that have been previously declared and defined using a mapping.

Using this concrete syntax, we now have all ingredients to show how the risks that we identified for the Payment subprocess can be captured via sensor conditions. There is an overtime process if an instance of the payment subprocess is not completed within an *MCT* of five days (see Section 2). This condition is checked by using two variables: one to retrieve the amount of time T_c expired and the other to retrieve the T_e remaining cycle time.

$d = 5$ $T_c = \text{case}(\text{current}).\text{Payment}(\text{PassTimeInMillis})$ $T_e = \text{case}(\text{current}).(\text{TimeEstimationInMillis})$

Assuming a tolerance value of 60%, the risk condition defined to monitor this risk is:

$$(1-(d*24*60*60*1000)/(T_e+T_c))>0.6.$$

There is an approval fraud whenever a Senior Finance Officer (SFO) approves two orders for the same customer within five days (see Section 2). Accordingly, the corresponding risk can be detected if given an order o of customer c to be approved, i) o is

allocated to a SFO who approved another order for the same customer in the last five days; or ii) the only SFOs available are the ones who approved an order for customer c in the last five days.

This risk condition is triggered by an event, i.e. the spawning of a new instance of task Approve Shipment Payment Order (see Figure 1). This is checked using a variable to retrieve the status of this task in the current instance. The risk condition itself is given by the disjunction of the two conditions described above. The first condition is checked by using variable $ASPO\#$ to retrieve the number of times this task was completed for customer c by the resources to whom is currently allocated. Variable $ASPO\#$ is defined via a case condition over customer c , the resource to whom is currently allocate (variable sfo_1), the completion time of this task (that must be greater than the start time of the current task Approve Shipment Payment Order minus five days in milliseconds²), and the identifier of the instance (that must be different from the identifier of the current instance).

The second condition is checked by using two variables and invoking two functions. Variable sfo_2 retrieves which resources completed task Approve Shipment Payment Order, and variable sfo retrieves all resources that can be offered this task (i.e. the current task). The first variable is defined via a case condition over customer c and the completion time of this task (that must be greater than the start time of the current task Approve Shipment Payment Order minus five days). The two invoked functions return the number of tasks

²Time is measured in milliseconds as logging systems save timestamps in this unit, and this facilitates the comparison of timings.

started on the resources that completed task Approve Shipment Payment Order, and the number of tasks in the execution queue of the resources that have been offered this task, and did not complete it for customer c in the last five days.

After the definition of the variables, defined in Table 1, the risk condition is specified as follows:

$$(ASPO\#>0)\vee((sfo_2.startMinNumber=0)\wedge(sfo.startMinNumberExcept.sfo_2\geq 1)).$$

An order unfulfillment occurs whenever an order is updated more than five times (see Section 2). Accordingly, the respective risk can be detected if: i) task Update Shipment Payment Order is currently being performed for a given order; and ii) it is likely that this order will be updated again (i.e. task Update Shipment Payment Order will be repeated within the same process instance). The probability that the number of times a task will be repeated within the same instance of the Payment subprocess is computed by dividing the number of instances, where five executions for task Update Shipment Payment Order has been reached, over the number of instances that have executed this task at least as many times as it has been executed by the current instance, and have completed. This condition can be checked by using variable $USPO\#US$ to retrieve the amount of orders that have been updated at least as much as this order, and variable $USPO\#U5$ to retrieve the amount of orders that have been updated at least five times.

The variables described can be defined via the sensor definition language as in Table 2. Assuming a tolerance value of 60%, the risk condition is specified as follows:

$$(USPO\#U5/USPO\#US)>0.6.$$

An underpayment fraud occurs whenever a customer underpays more than three times in a five-day time frame (see Section 2). Accordingly, the respective risk can be detected if: i) task Issue Debit Adjustment is being performed for a given customer and order (this is the trigger for this risk); and ii) the probability that the maximum number of allowable executions for this task will be reached in a five-day time frame, is above the fixed tolerance value for this risk, (this is the risk condition itself). This condition can be checked by using variable *Probability* to retrieve the probability that an attempted fraud will take place. This variable use the *Action* “*FraudProbabilityFunc*” to compute the specific probability (see Appendix B), and takes as input, among other information, variable $IDA\#Issue$ which retrieves the number of times task Issue Debit Adjustment has been completed for this customer.

The defined variables are implemented through the sensor language as follows as in Table 3. These variables are used to compose the following risk condition, assuming a tolerance value of 60%:

$$Probability>0.6.$$

The definition of actions and functions used in the above variables is provided in appendix. In particular, the complete list of all actions is provided in Appendix B, the complete lists of the nested loops and of the functions are provided in Appendix C and Appendix D.

7. Risk Templates

To facilitate the definition of concrete risk conditions on process models, we defined a

sfo ₁	=	case(current).Approve_Shipment_Payment_Order_593(allocateResource)
c	=	case(current).Issue_Shipment_Invoice_594.ShipmentInvoice.Company
d	=	5
ASPO	=	case(current).Approve_Shipment_Payment_Order_593(OfferTimeInMillis)
ASPO#	=	case(Approve_Shipment_Payment_Order_593(completeResource)=sfo ₁ ∧ Issue_Shipment_Invoice_594.ShipmentInvoice.Company=c ∧ Approve_Shipment_Payment_Order_593(CompleteTimeInMillis)> (ASPO-(d*24*60*60*1000)) ∧ (ID)!=IDCurr) .Approve_Shipment_Payment_Order_593(CountElements)
sfo ₂	=	case(Issue_Shipment_Invoice_594.ShipmentInvoice.Company=c ∧ Approve_Shipment_Payment_Order_593(isCompleted)="true" ∧ Approve_Shipment_Payment_Order_593(CompleteTimeInMillis)> (ASPO _{StartTime} -(d*24*60*60*1000)) ∧ (ID)!=IDCurr) .Approve_Shipment_Payment_Order_593(completeResource)
sfo	=	case(current).Approve_Shipment_Payment_Order_593(offerDistribution)

Table 1: Variable definition for approval fraud risk.

USPO#UC	=	case(current).Update_Shipment_Payment_Order_604(Count)
USPO#U5	=	case(Update_Shipment_Payment_Order_604(Count)>=5). Update_Shipment_Payment_Order_604(CountElements)
USPO#US	=	case(Update_Shipment_Payment_Order_604(Count)>=USPO#UC ∧ Process_Shipment_Payment_603(isOffered)="true"). Update_Shipment_Payment_Order_604(CountElements)

Table 2: Variable definition for order unfulfillment risk.

notion of risk template. A risk template is an “abstract” risk condition defined using generic tasks and generic variables associated with these tasks, which are not linked to any real process model. These generic tasks and variables will then be bound to real tasks and variables when the template is used to define a concrete risk condition for a specific process model. These templates are thus used as “shortcuts” to simplify the definition of risk conditions.

Several studies address different types of risks [76, 27, 62, 81, 86, 88]. After an analysis of these studies we decided to subdivide our risk templates into categories reflecting these types of risks. We expect that this division into categories also reduces the effort required by a user to select a suitable template. These categories were defined using the risk taxonomy proposed by

Rosemann and zur Muehlen [76] and other types of risks proposed in the literature [40]. Our organization of templates based on the above taxonomy is not intended to be an exhaustive coverage of all possible process-related risks. Instead, we provide an extensible structure that can easily be extended based on specific requirements or further taxonomies.

We organized risk templates in two levels. The first level is obtained using the risk taxonomy, obtaining five categories of risks. These categories are: i) *Structural*, capturing risks deriving from wrong decisions taken at design time; ii) *Data*, capturing risks of damaging data integrity; iii) *Technology*, capturing risks of impacting on the availability of IT systems; iv) *Organizational*, capturing risks of process faults that may impact on the employees or be caused

$IDA_{StartTime}$	=	case(current).Issue_Debit_Adjustment_605(StartTimeInMillis)
c	=	case(current).Issue_Shipment_Invoice_594.ShipmentInvoice.Company
d	=	5
$IDA_{\#Issue}$	=	case(Issue_Shipment_Invoice_594.ShipmentInvoice.Company=c $\wedge$ Issue_Debit_Adjustment_605(Count)>0 $\wedge$ Issue_Debit_Adjustment_605(CompleteTimeInMillis)>(IDA _{StartTime} -d*24*60*60*1000)) .Issue_Debit_Adjustment_605(CountElements)
GroupingElement	=	Issue_Shipment_Invoice_594.ShipmentInvoice.Company
WindowElement	=	Issue_Debit_Adjustment_605(CompleteTimeInMillis)
Threshold	=	0.6
Probability	=	case(Issue_Debit_Adjustment_605(Count)>0 $\wedge$ (ID)!=[IDcurr]). Issue_Debit_Adjustment_605(FraudProbabilityFunc, IDA _{#Issue} , 3, GroupingElement, WindowElement, (d*24*60*60*1000))

Table 3: Variable definition for underpayment fraud risk.

by them; and i) *Goal*, capturing risks of not achieving the process objectives that cannot be categorized in any of the previous categories. For each of these risk categories we defined eleven subcategories. These subcategories are based on the type of risks proposed in the literature [40]. Specifically, we have: i) *Strategic*, risks of affecting the implementation of business strategies; ii) *Operations*, risks of affecting the capability of supplying and producing services or goods; iii) *Supply*, risks that prevent a resource from executing an operation; iv) *Customer*, risks related to customers; v) *Asset*, risks deriving from the use of an asset; vi) *Competitive*, risks deriving from faults related to competitiveness; vii) *Reputation*, risks of loss of reputation; viii) *Financial*, risks of financial loss; ix) *Fiscal*, risks caused by changes in taxation; x) *Regulatory*, risks related to changes in regulations; xi) *Legal*, risks of faults that may lead to legal actions. Figure 6 shows a mind map illustrating how risk templates are categorized and subcategorized. This subdivision in categories is not absolute. Thus, a situation in which a risk could belong to more subcategories at the same time is possible. In this case the user should choose the template from the

subcategory considered to be the most appropriate, i.e. the one that would generate the risk condition that is the closest to the user requirements.

We defined 14 risk templates as a starting point for a larger risk template repository. For example, one template of type Data/Operations, aims to detect a possible inconsistency in the value of a critical variable. The template requires a as the critical variable, b the identifier variable, and two set of instances s_1 and s_2 , where s_1 is composed of the instances in which the values of a and b are equal to the value of a and b for the current instance, and s_2 is composed of the instances in which the value of a is equals to the value of a for the current instance. If the number of instances in s_1 divided by the number of instances in s_2 is greater than a certain threshold the template triggers a notification. This risk template can be used to represent a case in which the critical variable is a credit card number, or a bank account, while the identifier variable is the customer identifier.

Another risk template, of type Goal/Financial, detects the possibility that a particular process may exceed the budget assigned for its execution. This risk

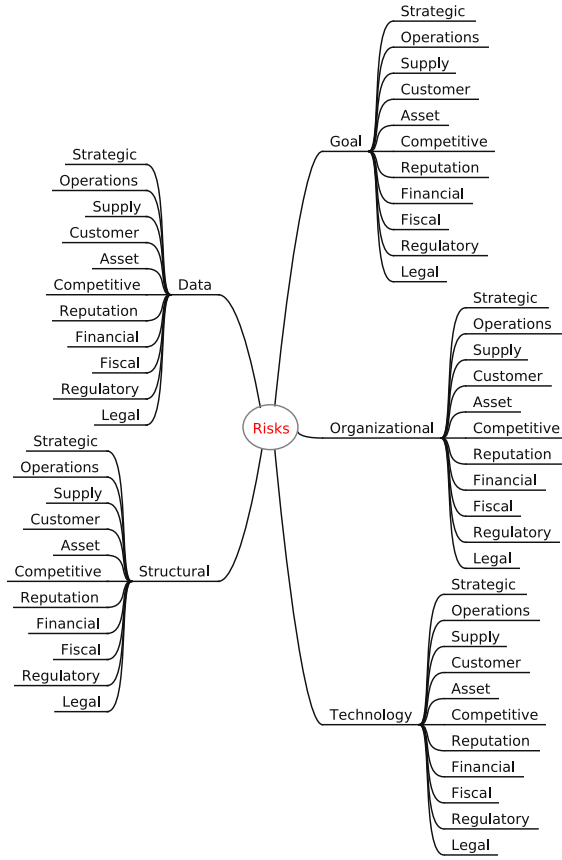


Figure 6: Template structure with categories and subcategories.

is detected looking at all the completed instances of the process and at all the completed instances with an execution cost at least equal to the execution cost of the current instance. The condition calculates the probability that the current instance will exceed the budget. This risk can be associated with any process where the cost is a relevant element. An example is an insurance process where opinions from different assessors may be required but the total cost of their involvement should not exceed the premium.

Considering the category Organizational, subcategory Reputation, we have risks that affect the reputation of the company due to

“employees’ mistakes”. For example, a delivery company may incur in a reputational risk if a delivery needs to be rescheduled due to an employee’ mistake. In this case the customer may think that the company is not professional enough with a consequent impact on the credibility of the company.

In the category Organizational, subcategory Supply, we defined a template that addresses possible delays with the execution of tasks. It detects the possibility that a critical task would not be started as soon as it is offered. This risk is detected looking at all the resources which have been offered a work item of the critical task. If the ratio between the number of resources that are busy versus the total number of resources is greater than a certain threshold the risk is detected. This type of risk is relevant for processes in healthcare such as a process for transferring organs from one hospital to another, where the unavailability of a resource may cause the organ deterioration. Finally, taking in consideration the process described in Section 2, let’s say that the company wants to avoid that processing a shipment payment may be subjected to delays, what we have to do is: i) import the template; ii) create a mapping of our generic critical task to the task “Process Shipment Payment”; and iii) modify the specified threshold if required.

Table 4 provides a brief overview of the templates currently available in the YAWL system (see Section 8). Besides the three templates described above, we have, among others, templates for the following risks: underpayment fraud, approval fraud, overtime for activity, sequence of activities and process, and violation of the four-eyes principle within and across cases.

Category	SubCategory	Template Name	Risk Description
Data	Financial	Underpayment Fraud	Underpayment fraud
	Operational	Data Inconsistency Across Cases	Wrong information provided, detected using multiple cases
		Data Inconsistency Parallel Activity	Wrong information provided to parallel tasks
Goal	Financial	Process Cost Exceeded	Exceeding the budget during the execution of the process
	Strategic	Activity Time Exceeded	Exceeding the time limit within which the activity needs to be completed
		Multi-Activity Time Exceeded	Exceeding the time limit within which a sequence of activities needs to be completed
		Process Time Exceeded	Exceeding the time limit within which the process needs to be completed
Organizational	Financial	Approval Fraud	Approval fraud
	Operational	Four-Eyes Principle	Four-Eyes Principle violation
		Four-Eyes Principle Across Cases	Four-Eyes Principle violation for activities across different cases
		Inadequate Preparation	The activity is executed by a novice resource
		Delay In Start	The activity start will be delayed
	Supply	Task Priority Unfulfilled	Activity with high priority cannot start because resources are busy
Structural	Operational	Loop	A loop is executed too many times

Table 4: Risk Templates and descriptions

8. Software Implementation

In order to prove the feasibility of our approach, we implemented the sensor-based architecture in the YAWL system.³ We decided to extend the YAWL system for the following reasons. First, this system is based on a service-oriented architecture, which facilitates the seamless addition of new services. Second, the system is open-source, which facilitates its distribution among academics and practitioners, and widely used in practice (the system has been downloaded over 100,000 times since its first inception in the open-source community). Finally, the underlying YAWL language is very expressive as it provides wide support for the workflow patterns [95].

As part of this implementation, we extended the YAWL Editor version 2.2beta with a new component, namely the Sensor Editor, for the specification of sensors within YAWL process models. Such graphical component, shown in Figure 7, fully sup-

ports the specification of sensor conditions as defined in Section 4, and the specification of risk templates.

A wizard (see figure 8) was developed as part of the YAWL Editor to facilitate the use of risk templates and in particular the mapping required for their use. This wizard using an user friendly interface guides the user during each step of the mapping, providing a description of each generic task and variable. The wizard also provides the possibility of customizing a risk, since it is possible to modify a risk condition during the creation of a risk using templates.

Moreover, we implemented the Sensor Manager as a generic component which exposes three interfaces (engine, database and monitor) as described in Section 4. We then wrapped this component into a Web service which implements the three interfaces for the YAWL system, allowing the component to interact with the YAWL Engine, the Monitor service and the YAWL database. While there is a straightforward mapping between the YAWL Engine and our engine

³Available at www.yawlfoundation.org

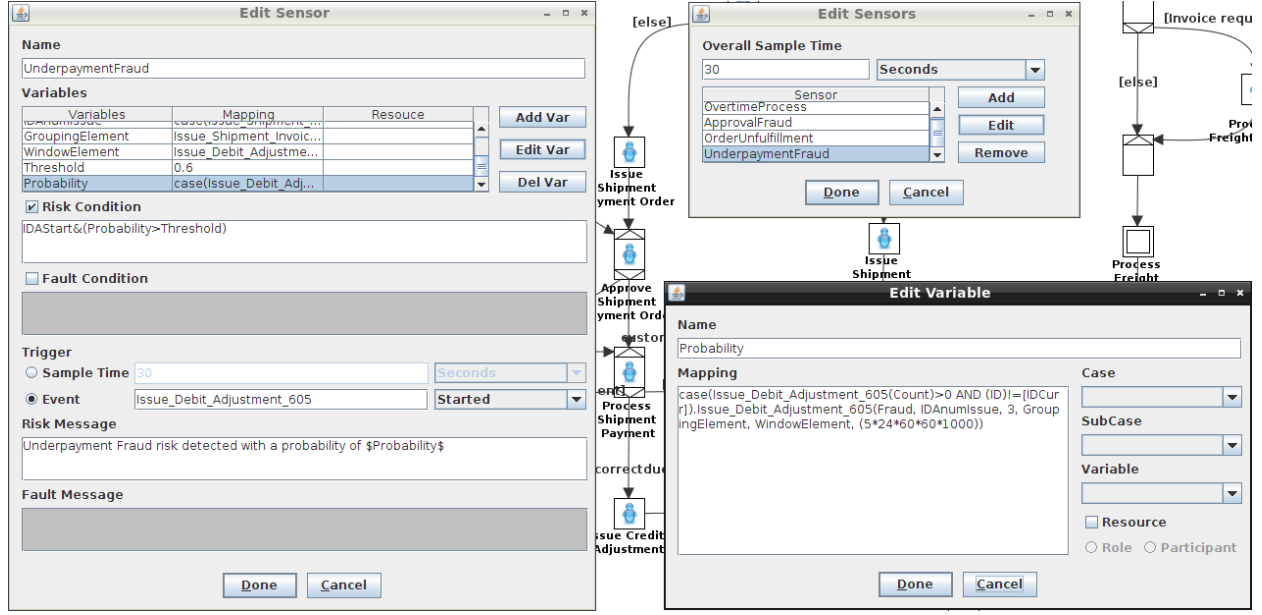


Figure 7: The Sensor Editor within the YAWL Editor.

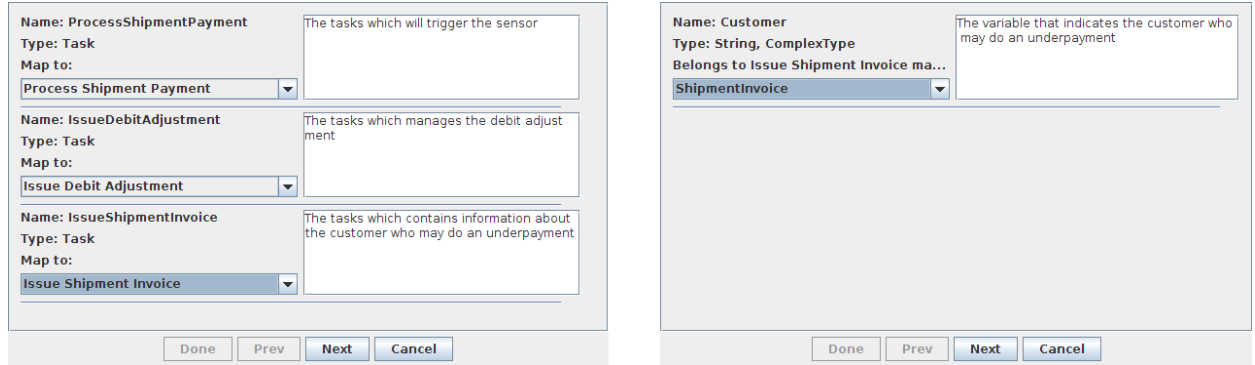


Figure 8: Template Wizard: a) Tasks mapping, b) Variables mapping.

interface, and between the YAWL Monitor service and our monitor interface, we had to join several YAWL tables to implement our database interface. This is because in the YAWL system, event logs are scattered across different database tables. For example, to retrieve all identifiers of the process instances for a specific process model, given the model identifier, we need to perform a join among the following YAWL tables: `logspecification`, `lognetinstance`, `lognet` and `logevent`.

The complete mapping is illustrated in Table 5. As an example, this table also shows the mapping between our database interface and the relational schema used by Oracle BPEL 10g to store BPEL process logs. Also in this case, the database can be fully mapped by joining several tables.

Finally, we implemented a separate service to estimate the remaining cycle time T_e for a process or task instance. This service uses ProM's prediction miner [1] to compute the estimations, and provides the re-

sults to the Sensor Manager on demand. While the estimation of T_e could be done on-line, i.e. while evaluating a particular sensor condition at run-time, parsing the full logset each time would be inefficient. Rather, we compute this estimation off-line, whenever a new process model is deployed to the YAWL Engine, by using the logset available at that time. Periodically, we update the logset with the new instances being executed meantime, and invoke this service to refresh the estimations for each process model currently deployed.

9. Evaluation

In this section we discuss two evaluations of the sensor-based architecture. In Section 9.1, we discuss a performance analysis of the implementation of the architecture in the YAWL system, and in Section 9.2, we report on a usability evaluation of the architecture on basis of an online focus-group with 21 students that gained experience in using the system for risk sensor condition analysis and definition tasks.

9.1. Performance Analysis

We used our implementation to evaluate the scalability of the approach. First, we measured the time needed to evaluate the basic functions (e.g. counting the number of instances of a task or retrieving the resource allocated to a task). Next, we measured the time needed to evaluate the sensor conditions for the risks defined in the Payment subprocess. The tests were run on an Intel Core I5 M560 2.67GHz processor with 4GB RAM running Linux Ubuntu 11.4. The YAWL logs were stored on the PostGres 9.0 DBMS. These logs contained 318 completed process instances from 36 difference process

models, accounting for a total of 9,399 process events (e.g. task instance started and completed, variable’s value change). Specifically, there were 100 instances from the Payment subprocess yielding a total of 5,904 process events. The results were averaged over 10 runs.

Table 6 shows the results of the evaluation of the basic functions provided by our language. In particular, in this table we compare the evaluation times obtained by accessing the YAWL logs via our database interface, with those obtained by accessing a serialization of the logs, e.g. in the OpenXES format. While OpenXES provides a simple and unique representation of a generic set of process logs, accessing an OpenXES file in real-time, i.e. during the execution of a process instance, is not feasible, due to the long access times (e.g. 6.5 sec. on average for evaluating a net variable). On the other hand, accessing the logs via our database interface, despite it requires the creation of a specific implementation for each BPMS database, provides considerably faster times than accessing OpenXES files (at least 87% gain w.r.t. OpenXES access). In fact, as we can see from Table 6, the evaluation times for all the basic functions are below 30 ms, apart from function `task variable`, which takes almost 100 ms and function `net variable`, which takes about 430 ms.

The last two basic functions reported in Table 6, namely `task distribution` and `task initiator`, are evaluated in less than 250 milliseconds. These functions are not computed by accessing the logs, but rather by accessing information that is contained directly in an executable process model, e.g. the resources that are associated with a specific task. However, in our implementation we still use the database interface to ac-

Database table	Tables that need to be joined	
	YAWL	Oracle BPEL 10g
WorkFlowDefinition	logspecification, lognet, lognetinstance, logevent	cube.instance and cube.scope
SubProcess	logspecification, lognet, lognetinstance, logevent	cube.instance and cube.scope
Activity	lognetinstance, logtask, logtaskinstance, lognet, logevent, logspecification, rs_eventlog	wftask and work_item
Variables	logtask, lognet, lognetinstance, logtaskinstance, logevent, logdataitem, logspecification	audit_trail, audit_detail and xml_document
Role	rs_participant	wftask
ActivityRole	rs_eventlog, logtaskinstance	wftask

Table 5: Database interface mapping for YAWL 2.2beta and Oracle BPEL 10g.

Basic function	Description	OpenXES time [ms]	Database time [ms]	Reduction rate [%]
net status	functions checking if a net status has been reached (isStarted, isCompleted)	6,535	18.9	99.71
net time	functions returning the time when a net status has been reached (startTime, completeTime, startTimeInMillis, completeTimeInMillis)	6,781	18.8	99.72
net variable	returns the value of a net variable	6,489	432.6	93.33
task count	number of times a task has been completed	803	19.8	97.53
task resource	functions that return the resources associated with a task (offerResource, allocateResource, startResource, completeResource)	850	20.9	97.54
task status	functions checking if a task status has been reached (isOffered, isAllocated, isStarted, isCompleted)	792	30.5	96.14
task time	functions returning the time when a task status has been reached (offerTime, allocateTime, startTime, completeTime, offerTimeInMillis, allocateTimeInMillis, startTimeInMillis, completeTimeInMillis)	824	22.3	97.29
task variable	returns the value of a task variable	787	96.7	87.71
task distribution	functions returning the resources associated with a task by default (offerDistribution, allocateDistribution, startDistribution, completeDistribution)	243		-
task initiator	functions returning the allocation strategy for a resource association (offerInitiator, allocateInitiator, startInitiator, completeInitiator)	249.6		-

Table 6: Performance of basic functions.

cess this information, in order to provide the developer with a single access point to all process-related data.

Table 7 reports the results of the evaluation of the sensor conditions defined for our running example. While the sensor conditions for the overtime process and order unfulfillment faults are very low (below 150 ms), longer times are obtained for evaluating the conditions for the two faults related to fraud. This is because both these conditions require to evaluate “complex queries”, i.e. queries over the entire process logs: in the approval fraud, we need to retrieve all

resources that approved an order for a specific customer, while in the underpayment fraud we need to retrieve all process instances where a debit adjustment was issued and aggregate these instances per customer. These queries are different than those needed to evaluate the basic functions, as the latter are performed on the events in the logs that are relative to a single known process instance, e.g. the instance for which the sensor condition is being evaluated.

The worst-case complexity of evaluating one such a complex query is still linear

Sensor	Min [ms]	Max [ms]	Ave [ms]	St.Dev.
Overtime process	121	137	131.8	4.66
Approval fraud	6,483	7,036	6,766.4	183.06
Order unfulfillment	69	91	77.4	7.18
Underpayment fraud	3,385	3,678	3,523	89.98

Table 7: Performance of sensors.

on the number of parameters that need be evaluated in the query (corresponding to the language element *CondExprSet* in Section 4) multiplied by the total number of instances present in the logs (corresponding to the size of table *WorkflowDefinition* addressed by our database interface).

In conclusion, the performances of evaluating sensor conditions should always be considered w.r.t. the specific process for which the risks are defined, and the type of trigger used. For example, let us assume an average duration of 24 hours for the Payment subprocess, with a new task being executed every 30 minutes. This means we have up to 30 minutes to detect an overtime process risk before a new task is executed, and we need to compute this sensor condition again. If we choose a rate of 5 minutes to sample this condition, we are well below the 6 minute-threshold, so we can check this sensor condition up to 6 times during the execution of a task. Since we do this in less than 150 ms, this time is acceptable. For an event-driven risk we also need to consider the frequency of the specific event used as trigger. For example, the approval fraud risk is triggered every time an instance of task Approve Shipment Payment Order is offered to a Senior Financial Officer for execution. Since we take up to 7 seconds to compute this sensor condition, we are able to cope with a system where there is a request for approval every 7 seconds. So also for this sensor, the performance is quite ac-

ceptable.

9.2. Usability Analysis

For the evaluation of the usability of the sensor-based architecture, we followed the example from related studies [54, 55] and conducted focus group-like online sessions with students. Each online session consisted of a tutorial followed by a questionnaire. Specifically, the tutorial provided detailed instructions on the developed component for defining sensors, and then presented two tasks that the students had to perform with the system, related to the analysis of risk conditions and of risk sensor conditions. Once the students completed the tutorial, they had to report on their usage experiences using a structured questionnaire (see Appendix E). The sessions were not timed and no time limits were imposed on the users.

The questionnaire consisted of five parts:

- In part one, participants had to provide relevant demographic information about their process modeling experience.
- In part two, they were asked to provide details about their experience in using workflow management systems in general.
- In part three, they were asked to provide details about their experience in using YAWL in particular. The questions for parts one to three were, where possible, adapted from prior surveys on process modeling [54, 55] and usage of the YAWL system [74].
- In part four, users had to report on their experience in using the system to perform the two tasks in the tutorial. Specifically, these were:

1. the analysis of the risk sensor conditions for two YAWL models using the developed risk definition language (risk scenarios 1 and 3 in Appendix E), and
2. the specification of risk sensor conditions for a YAWL model based on a risk definition using fault tree analysis (risk scenario 2 in Appendix E).

The tasks in the tutorial were presented using three risk scenarios on the basis of the developed sensor-based architecture. Risk scenarios 1 and 3 describe risks using the developed risk definition language. Here we asked participants to analyze the risk proposed. Risk scenario 2 describes a risk using a fault tree. Here we asked participants to define such a risk using the proposed language.

For each of these application tasks, we assessed how well participants were able to assess risks, in terms of *accuracy* of understanding the risks and the *difficulty* of understanding the meaning of the risks. Accuracy and difficulty are widely used measures [14, 60] of the effectiveness and the efficiency of the sensor-based approach in terms of how well users of the approach can understand the risk information provided through the approach, and how much cognitive effort is required to develop this understanding. We used a 5-point scale to measure the difficulty of understanding the risk from very simple to very difficult, and we defined five true/false/do not know questions to measure accuracy of comprehension of the risk information provided.

- In part five, we captured participants' perceptions about the usage experience of the approach as implemented in the YAWL environment. This part of the questionnaire contained measurement items that were adapted from [74] and measured satisfaction (SAT), usefulness (PU), ease of use (PEOU) and intentions to use (ITU). Similar to previous uses of these items [71, 72, 74], the measures were of appropriate reliability, with Cronbachs Alphas ranging from 0.64 (ITU) to 0.81 (PU).

For this evaluation, we recruited participants by contacting individuals that had previously completed a university undergraduate or post-graduate course on business process automation. The cohorts were from the Università della Calabria, Cosenza, Italy and the Queensland University of Technology, Brisbane, Australia, and we perused the course email lists for contact. We recruited participants from these cohorts because principles of process modeling and automation in general as well as the YAWL system specifically were featured in the courses, thus allowing us to select participants with sufficient levels of background experience and expertise, in turn minimizing the risk of confounding the usability results due to lack of experience with workflow systems or process modeling in general.

The usability test was administered between January and September 2012. Overall 21 participants participated, where 10 of them were from Italy and 11 from Australia. Participants, on average, had about 2.6 years of experience with process modeling and had read and created, on average, 71 and 15 process models, respectively, over the last twelve months. Participants' experience with different process

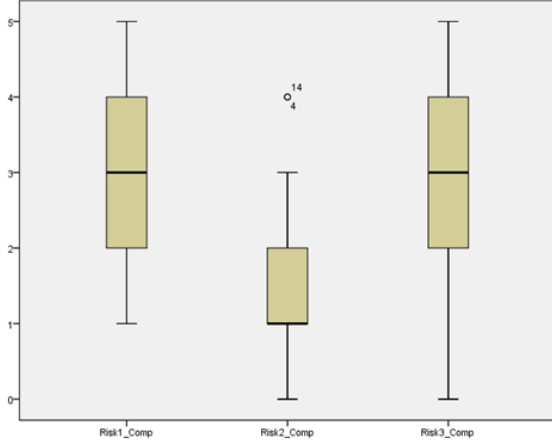


Figure 9: Accuracy of risk comprehension, on a scale from 0 (low) to 5 (high).

modeling languages varied, with most having experience, at least, with UML Activity Diagrams, Petri Nets, EPCs and/or BPMN. These characteristics describe our participants as proxies for novice to average BPM professionals, with one of our participants representative of an expert practitioners (more than 5 years experience, more than 250 models created, more than 1000 models read). Overall, our study population might not be representative of experts but is roughly equivalent to the range of expertise found in actual BPM practitioners as reported in other surveys, e.g. [73].

The box-plots in Figure 9 and Figure 10 show the accuracy of the comprehension of the risks in the three scenarios as well as the perceived difficulty of understanding these risks. The data shows that scenarios 1 and 3 were reasonably well understood (averages for the comprehension questions were 2.9 for scenario 1 and 3.1 for scenario 3, both exceeding 50% accuracy), while comprehension of scenario 2 was significantly lower (average score 1.7). Similarly, perceived difficulty was highest for scenario 2 (average of 3.7), while the perceived difficulty for sce-

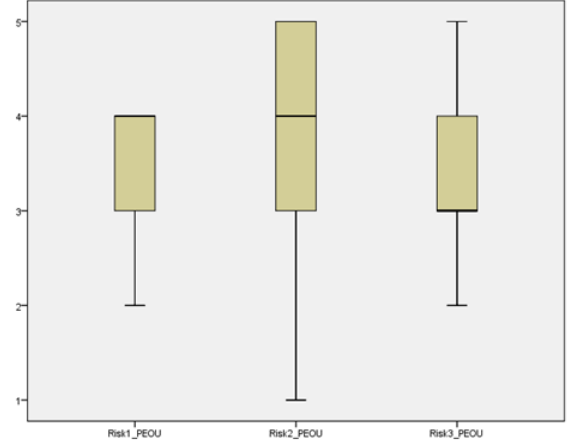


Figure 10: Difficulty of risk comprehension, on a scale from 1 (very simple) to 5 (very difficult).

nario 1 and 3 was similar in range (averages 3.4 and 3.3, respectively).

Next, we were interested in understanding under which circumstances participants were able to obtain higher levels of accuracy in understanding the risk information provided. To that, we built a regression model in which we regressed relevant demographic data and perceived difficulty onto the total comprehension score across all three risk scenarios. Specifically, we included as coefficients:

- PMExp(Years): Process modeling experience in years
- PMExp(Training): Extent of formal training in process modeling in days over the last twelve months
- PMExp(SelfEducation): Extent of self-education in process modeling in days over the last twelve months
- PMExp(processModelsCreated): Number of process models created over the last twelve months

Variable	Standardized Coefficients		
	Beta	t	Sig.
PMExp(Years)	-0.80	-2.79	0.02
PMExp(Training)	-0.03	-0.16	0.88
PMExp (processModelsCreated)	1.28	4.31	0.00
BPMSFAM	0.38	1.15	0.28
YAWLUse(time)	0.80	3.07	0.01
YAWLUse(features)	-0.30	-1.40	0.20
YAWLUse(models)	-1.20	-3.86	0.00
RISKPerDif	-0.19	-1.05	0.32

Table 8: Regression analysis of risk comprehension across all three scenarios

- BPMSFAM: Self-perceived familiarity with BPMSs [71]
- YAWLUse(time): length of use of the YAWL system
- YAWLUse(features): Number of YAWL features used
- YAWLUse(models): Number of YAWL models created, read or edited over the last twelve months
- RISKPerDif: Average perceived comprehension difficulty across the three presented scenarios.

The regression model was significant ($F = 3.54$, $p = 0.04$) and explained 77.9 percent of the variance in total comprehension score, thus attesting to very good explanatory power. Table 8 gives the results from the regression model analysis.

Table 8 shows that four factors were significant predictors for explaining comprehension accuracy, these being process modeling experience in years, number of process models created, use of the YAWL system and number of YAWL models created, edited or read over the last twelve months. Several interesting findings are noteworthy. First, PMExp(Years) and YAWLUse(models) are negative predictors,

which may suggest that novice users with little process modeling experience and little experience with YAWL models benefited more from the risk sensor approach. Second, the difficulty of understanding the risk information provided is not related to how well users understand the risks. Third, training in process modeling or more varied use of the YAWL system, likewise, are irrelevant in terms of understanding risk information provided by the sensor-based architecture.

Finally, we were interested in the participants’ evaluation of the sensor-based architecture. Following the theory of technology acceptance [28] and its extended application in the process modeling context [72], we understand that satisfaction, ease of use and perceived usefulness are key criteria for explaining intentions to use a process modeling artifact. Figure 11 shows the box-plots for the average total factor scores for these four criteria.

The data displayed in Figure 11 suggests that participants rated the usefulness of the approach high (mean score > 5.3) and were in general inclined to use the system (mean score > 4.4). Satisfaction with the approach was reported as average (mean score = 4.0) and ease of use, notably, was reported as low (mean score < 3.5), indicating potential to improve interface and user interaction of the system. This is not surprising, since we did not explicitly consider the principle of “clarity” when designing the concrete syntax of our language.

In a post-hoc analysis, we compared evaluations of SAT, PU, PEOU and ITU across users that scored low or high on risk comprehension, based on a median split, and across users that rated the perceived difficulty of understanding risks as low or high, again based on a median split. MANOVA

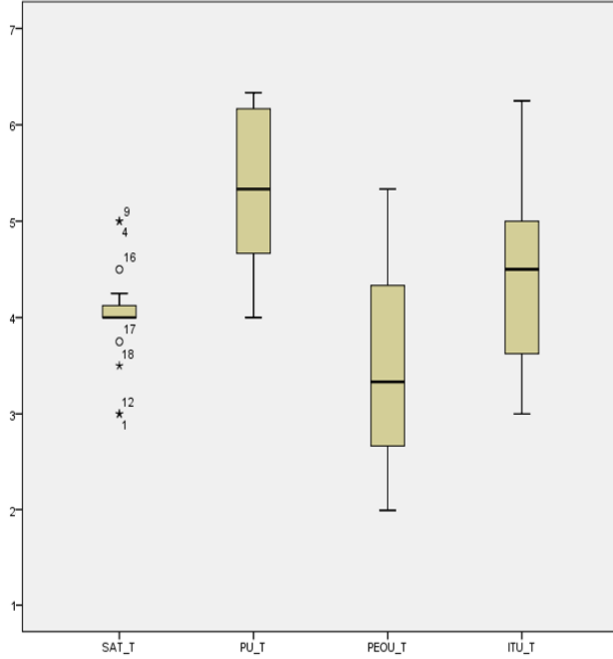


Figure 11: Perceptual evaluations of the sensor-based architecture on a scale from 1 (low) to 7 (high).

tests in both cases showed that evaluations of the approach were independent from comprehension accuracy or difficulty, with p-values ranging from 0.30 to 0.49 and 0.18 to 0.58, respectively. The results indicate that the evaluations of satisfaction, usefulness, ease of use and intentions to use were robust against variances in the performance in using the system.

In summary, our evaluation showed that the sensor-based risk identification and modeling approach provided value in allowing participants to develop an understanding of process risks. We found the approach to be useful particularly for users with limited experience in process modeling or BPMSs. The evaluation further revealed that ease of use of the system should be improved to warrant better user acceptance.

10. Related Work

Risk measurement and mitigation techniques have been widely explored in various fields. At the strategic-level, risk management standards prescribe generic procedures for identifying, analyzing, evaluating and treating risks (see e.g. [89]). Although helpful, such general guidelines are inevitably vague and fail to provide any specific guidance for operationalizing risk management strategies in business processes. At the other extreme, there are many techniques for identifying risks in specific areas such as employee [3], conflict of interest [57] and in the engineering field more generally [43, 13]. Other approaches, such as fault-tree analysis [21], are general enough to be applied to multiple domains. However, none of these approaches provides insights on how to define and operationalize the detection of process-related risks. In the following, we first discuss related work at methodological level with respect to the proposed approach and then related work with respect to the architectural level of our proposal is discussed.

Previous process-based research recognizes the importance of explicitly linking elements of risk to business process models, through specific methodological approaches. Such approaches, discussed below, can be mainly categorized into design time Risk-aware BPM with and without integrated risks constructs. The former [76, 102, 48, 97, 82, 85, 83, 84, 26, 25, 100, 5] analyzes and models BPM risks through the introduction of new integrated risk constructs whereas the latter [64, 67, 78, 77, 11, 42, 91, 53, 94, 70, 56, 12, 46, 6, 7, 34, 35, 51, 4, 10, 80, 49, 87, 52] reuses existing risk analysis methods. In the next paragraphs we are going to discuss the most relevant of these ap-

proaches while we refer to [92] for a comprehensive discussion on all these approaches.

Among these approaches we can identify several groups. The first group is composed of the approaches described in [26, 25, 100, 64, 67, 78, 77, 82, 85, 83, 84, 56]. They propose to extend existing modeling languages, such as Business Process Model and Notation (BPMN), Event-driven Process Chain (EPC), Value-Focused Process Engineering (VFPE), semantic business process modeling language (SBPML) and the integrated definition (IDEF) language, with risk-related constructs. The approaches provide the possibility to annotate business process models with risk-related information and in some cases it is also possible to annotate mitigation actions. Furthermore Rotaru et al. [67, 78, 77] also propose an utility calculation technique that can be used to determine optimal risk countermeasure solutions.

The second group is composed of approaches [4, 10, 80, 11, 6, 7] which propose risk-informed design. In these approaches common is the idea of modelling business processes (in one or several stages) which contain elements of risk and possible mitigation actions. These models are then used to create a process model in which mitigation activities are already part of the process itself. In particular, Betz et al. [11] propose a method based on simulation to choose the optimum process model variant if several variants are proposed.

The third group focuses on simulation and is composed of the approaches described in [48, 97, 94, 51, 49]. The ROPE (Risk-Oriented Process Evaluation), methodology proposed by Jakoubi et al. [48, 97], is based on the observation that faults (here called “threats”) impact the functionality of resources (required for the execution of pro-

cess activities). If a threat is detected, a countermeasure process or a recovery process is invoked to counteract the threat. The ROPE methodology aims to incorporate these aspects in a single model that can be simulated to determine a company's critical business processes and single points of failure. Similarly, Taylor et al. [94], propose a simulation environment based on the jBPM Process Definition Language (JPDL) workflow language. In this environment a process models annotated with risk information (i.e. key risk indicator (KRI), key performance indicator (KPI), and risk event) can be simulated in order to evaluate the effects of risk events on some predefined KPIs and KRIs. Finally, Kaegi et al. [51] simulate a process model described in BPMN via agent-based modelling technique to analyze business process-related risks, while Jallow et al. [49], propose an approach where risks in business processes are analysed. The approach use the Monte Carlo simulation [61], on a given a set of identified risk events and their occurrence probabilities, in order to assess and quantify the impact/consequences of those risk events (in terms of time, cost, performance, and other objectives) on each process activity and on the overall process.

Other works which cannot be grouped together annoverate: i) a taxonomy of process-related risks [76, 102, 67], which includes five process-related risk types (goals, structure, information technology, data and organization) that can be captured by four interrelated model types (risk structure model, risk/goal matrix, risk state model, and an extension to the EPC notation); ii) an extended goal-risk framework [5], which consists of an asset layer (composed of business process goals, activities, and business artifacts), an event layer (composed of

various events, including risk events, that can impact the asset layer), and a treatment layer (composed of a set of risk treatment activities that can mitigate the impact of the risk events modeled in the event layer); and iii) a technique to evaluate a workflow’s non-completion risk due to uncertain/dynamic information [87], which quantifies the confidence level of the non-monotonic predicates of a workflow and whether one of these confidence levels is below certain threshold considers the workflow to be risky suggesting the use of a backup workflow.

Finally, in the approach proposed by Kang et al. [52], a technique to estimate the probability that a process instance enters an abnormal termination state is defined. Process-related historical data is used to inform the probability estimation calculation. Then, a run-time risk estimation algorithm is developed such that appropriate risk alerts can be produced when risky situations are detected. The main limitation of this approach is the necessity of having an exhaustive log containing all possible executions and the impossibility of making distinctions among different abnormal termination states.

With respect to the risk-aware BPM life-cycle shown in Figure 3, all the above proposals cover the risk-aware process modeling phase (see Table 9). None of them specifies how risk conditions can be concretely linked to run-time aspects of process models such as resource allocation, data variables and control-flow conditions, for the sake of detecting risks during process execution. Thus, none of these approaches operationalizes risk detection into workflow management systems. Moreover, they neglect historical process data for risk estimation. As such, these approaches are complemen-

tary to our work, i.e. they can be used at a conceptual level for the identification of process-related risks, which can then be implemented via our sensor-based lower-level methodology.

From the architectural point of view, our approach, specifically our sensor-based architecture, is also related to real-time monitoring of business process execution. Similarly to our approach, Oracle Business Activity Monitoring (BAM) [69] relies on sensors to monitor the execution of BPEL processes. Three types of sensors can be defined: activity sensors, to grab timings and variable contents of a specific activity; variable sensors, to grab the content of the variables defined for whole BPEL process (e.g. the inputs to the process); and fault sensors, to monitor BPEL faults. These sensors can be triggered by a predefined set of events (e.g. task activation, task completion). For each sensor, one can specify the endpoints where the sensor will publish its data at run-time (e.g. a database or a JMSQueue). We allow the specification of more sophisticated sensor (and fault) conditions, where different process-related aspects can be incorporated such as data, resource allocation strategies, order dependencies, as well as historical data and information from other running process instances. Moreover, our sensors can be triggered by process events or sampled at a given rate. Nonetheless, our sensor-based architecture is exposed as a service and as such it could be integrated with other process monitoring systems, such as Oracle BAM.

A *Sensor*, as defined in this work, may be categorized as a specialization of the more general rules framework known as *Event-Condition-Action* (ECA) rules, which originally came to prominence through their use with Active Database Management Sys-

Approach	Process design/ Risk-aware process modeling	Process implementation/ Risk-aware workflow implementation	Process enactment/ Risk-aware workflow execution	Process diagnosis/ Risk monitoring
zur Muehlen et al. [76, 102]	✓			
Asnar and Giorgini [5]	✓			
Karagiannis et al. [53]	✓			
Singh et al. [87]	✓			
Cope et al. [26, 25]	✓			
Weiss and Winkelmann [100]	✓			
Mock and Corvo [64]	✓			
Rotaru et al. [67, 78, 77]	✓			
Sienou et al. [82, 85, 83, 84]	✓			
Lambert et al. [56]	✓			
Jakoubi et al. [48, 97]	✓			
Taylor et al. [94]	✓			
Kaegi et al. [51]	✓			
Jallow et al. [49]	✓			
Bergholtz et al. [4, 10, 80]	✓			
Betz et al. [11]	✓			
Bhuiyan et al. [6, 7]	✓			
Hermann and Hermann [42]	✓			
Strecker et al. [91]	✓			
Panayiotou et al. [70]	✓			
Bhuiyan et al. [12, 46]	✓			
Fenz et al. [34, 35]	✓			
Kang et al. [52]			✓	
Our proposal	✓	✓	✓	✓

Table 9: Comparison of available R-BPM approaches with respect to the four phases of our R-BPM lifecycle.

tems [30]. A Sensor extends the ECA notion by including two conditions, one which describes a potential *fault*, and the other the possibility of *risk*. A Sensor also delivers a consequence when a triggered condition evaluates to true, rather than an action *per se*. ECA rules have also been used as the basis for several well-known exception handling strategies in workflow systems [15, 17, 19]. For example, a definition language for exception rules called *Chimera-Exec* was developed for the *WIDE* prototype [16]. An extended ECA framework can be found in the *defeasable* workflow approach [59], which has an added *justification* condition to support context-dependent reasoning of actions to be taken. The *RUMBA* project combines ECA rules with an Aspect-Oriented framework to provide a modular approach to the integra-

tion of rules and active processes [18]. The *Worklet* approach incorporates an extended rules-based framework, which embeds exceptions directly into the evaluation tree [2].

Real-time monitoring of process models can also be achieved via Complex Event Processing (CEP) systems. In this context, CEP systems have been integrated into commercial BPMSs, e.g. webMethods Business Events⁴, ARIS Process Event Monitor [29] and SAP Sybase [93], as well as explored in academia [36, 41]. A CEP system allows the analysis of aggregated events from different sources (e.g. databases, email accounts as well as process engines). Using predefined rules, generally defined with a specific SQLlike language [99], a CEP system can verify the presence of a specific

⁴<http://www.softwareag.com/au/products/wm/events/overview>

pattern among a stream of simple events processed in a given time window. Our approach differs from CEP systems in the following aspects: i) strong business process orientation vs general purpose system; ii) ability to aggregate and analyze complex XML-based events (e.g. process variables) vs simple events; iii) time-driven and event-driven triggers vs event-driven trigger only. Moreover, CEP systems typically suffer from performance overheads [41, 99] which limit their applicability to real-time risk detection [99].

This article is an extended version of the work presented in [23]. Compared to this work, this article provides the full and revised definition of the language’s abstract syntax, the support for risk templates, the usability evaluation with users and a comprehensive related work.

11. Conclusion

We contributed an approach for real-time monitoring of risks in executable business process models. The approach embeds elements of risk into each phase of the BPM lifecycle: from process design, where high-level risks defined via a risk analysis method are mapped down to specific process model elements such as activities, resources and data, through to process diagnosis, where risks are detected during process execution, and those no longer tolerable are notified to process administrators. To the best of our knowledge, this is the first attempt to concretely embed risks into executable business processes and enable their automatic detection at run-time.

As a second contribution, we provided an operationalization of the proposed risk-awareness approach on top of BPMSs. This is achieved via a distributed, sensor-

based architecture that communicates with a BPMS via a set of tool-independent interfaces. Each risk is associated with a sensor condition and refers to a fault, which is an undesired state of the process. Conditions can relate to any process aspect, such as control-flow dependencies, resource allocations, the content of data elements, both from the current process instance and from instances of any process that have already been completed. At design-time, these conditions are expressed within a process model via a simple query language, for which we provide an abstract syntax. At run-time, each sensor independently alerts a sensor manager when the associated risk condition evaluates to true during the execution of a specific process instance. When this occurs, the sensor manager notifies a process administrator about the given risk by interfacing with the monitoring service of the BPMS. This allows early risk detection which in turn enables proper remedial actions to be taken in order to avoid potentially costly process faults.

We designed a set of risk templates to allow process designers to easily specify new risk conditions into a process model. Each template captures an abstract risk. To use these templates, one has to bind the template variables to concrete elements of the process model for which the risk condition needs to be monitored. We contend that by using such templates the effort of defining risks in executable process models can be reduced.

As a proof-of-concept, we implemented the sensor-based architecture on top of the YAWL system along with 14 representative templates. We then used the tool to evaluate the feasibility of the approach in practice. This was carried out in two directions. First, we evaluated the per-

formance of the implementation; second, we evaluated the usability and ease of use of the approach with users of the YAWL system. The performance measurements showed that the sensor conditions can be computed efficiently and that no performance overhead is induced to the BPMS engine. The results of the empirical evaluation with users showed that the sensor-based risk identification and modeling approach provided value to develop an understanding of process risks. We found the approach to be useful particularly for users with limited experience in process modeling or BPMSs. The evaluation further revealed that ease of use of the language for defining sensors should be improved to warrant better user acceptance. In order to overcome this limitation we plan to devise a mechanism for automatically deriving skeletons of sensors directly from fault trees. We expect that this will allow a smoother transition from high-level risk definitions to low-level risk conditions. Moreover, aiding features provided by common source code editors such as autocomplete, syntax highlighting and bracket matching can be put in place to reduce the human effort.

The approach presented in this paper and its operationalization serve as the cornerstone for other techniques that aim to bridge the gap between risk and process management. In particular, in [24] we documented a technique which uses input from this approach in order to mitigate risks. Specifically, as soon as one or more risks are detected which are no longer tolerable, the technique identifies a set of alternative mitigation actions that can be applied by process administrators. A mitigation action is a sequence of controlled changes applied to the process instance, that takes into account a snapshot of the process re-

sources and data, and the current status of the system in which the process is executed. Furthermore, in [22] we reported on a second technique which also builds on the approach presented in this paper to allow process participants to make risk-informed decisions. This is achieved by estimating the risk that the current process instance will end up with one or more faults based on the input provided by the participant, e.g. when filling out a user form based on the conditions specified for each fault.

This work suffers from several limitations which offer opportunities for future work. First, to overcome the low perceived ease of use of the tool resulting from the usability tests, we plan to improve the concrete syntax of the sensor definition language, taking into account the design principle of clarity. Second, manual work is currently required to convert the results of risk analysis into sensor conditions. This operation is error-prone, especially in the content of complex risk definitions. In this respect, we plan to devise a mechanism for automatically deriving skeletons of sensor conditions directly from a fault tree. This will allow a smoother transition from high-level risk definitions to low-level sensor conditions. Finally, we plan to evaluate the usefulness of templates with end users. In doing so, we will consider the results against their correlation with the user background in terms of exposure to YAWL in particular, and to risk management in general.

Acknowledgments We thank Peter Hughes for his help with the identification of process-related risks. This research is partly funded by the ARC Discovery Project “Risk-aware Business Process Management” (DP110100091). NICTA is funded by the Australian Government

as represented by the Department of Broadband, Communications and the Digital Economy and the Australian Research Council through the ICT Centre of Excellence program.

References

- [1] van der Aalst, W.M., Schonenberg, M., Song, M., 2011. Time prediction based on process mining. *Information Systems* 36, 450 – 475.
- [2] Adams, M., ter Hofstede, A., van der Aalst, W., Edmond, D., 2007. Dynamic, extensible and context-aware exception handling for workflows, in: *Proceedings of the 15th International Conference on Cooperative Information Systems (CoopIS'07)*, Springer, Vilamoura, Portugal. pp. 95–112.
- [3] Albrecht, W.S., Albrecht, C.C., Albrecht, C.O., Zimbelman, M.F., 2008. *Fraud Examination*. 3rd ed., South-Western Publishing.
- [4] Andersson, B., Bergholtz, M., Edirisuriya, A., Ilayperuma, T., Johannesson, P., 2005. A declarative foundation of process models, in: Pastor, O., e Cunha, J.F. (Eds.), *CAiSE*, Springer. pp. 233–247.
- [5] Asnar, Y., Giorgini, P., 2008. Analyzing business continuity through a multi-layers model, in: Dumas, M., Reichert, M., Shan, M.C. (Eds.), *BPM*, Springer. pp. 212–227.
- [6] Bagchi, S., Bai, X., Kalagnanam, J., 2006. Data quality management using business process modeling, in: *IEEE SCC*, IEEE Computer Society. pp. 398–405.
- [7] Bai, X., Padman, R., Krishnan, R., 2007. A risk management approach to business process design, in: *ICIS*, Association for Information Systems. p. 28.
- [8] Basel Committee on Bankin Supervision, 2006. *Basel II - International Convergence of Capital Measurement and Capital Standards*.
- [9] Bentley, J., 1986. Programming pearls: little languages. *Communications of the ACM* 29, 711–721.
- [10] Bergholtz, M., Grégoire, B., Johannesson, P., Schmitt, M., Wohed, P., Zdravkovic, J., 2005. Integrated methodology for linking business and process models with risk mitigation, in: *REBNITA*, Citeseer.
- [11] Betz, S., Hickl, S., Oberweis, A., 2011. Risk-aware business process modeling and simulation using xml nets, in: Hofreiter, B., Dubois, E., Lin, K.J., Setzer, T., Godart, C., Proper, E., Bodenstag, L. (Eds.), *CEC*, IEEE. pp. 349–356.
- [12] Bhuiyan, M., Islam, M.M.Z., Koliadis, G., Krishna, A., Ghose, A., 2007. Managing business process risk using rich organizational models, in: *COMPSAC (2)*, IEEE Computer Society. pp. 509–520.
- [13] Bhushan, N., Rai, K., 2004. *Strategic Decision Making: Applying the Analytic Hierarchy Process*. 3rd ed., Springer.
- [14] Burton-Jones, A., Wand, Y., Weber, R., 2009. Guidelines for empirical evaluations of conceptual modeling grammars. *Journal of the Association for Information Systems* 10, 495–532.
- [15] Casati, F., 1999. A discussion on approaches to handling exceptions in workflows. *ACM SIGGROUP Bulletin* 20, 3–4.
- [16] Casati, F., Fugini, M., Mirbel, I., 1999. An environment for designing exceptions in workflows. *Information Systems* 24, 255–273.
- [17] Casati, F., Ilnicki, S., Jin, L., Krishnamoorthy, V., Shan, M.C., 2000. Adaptive and dynamic service composition in eFlow , 13–31.
- [18] Cetin, S., Altintas, N., Solmaz, R., 2006. Business rules segregation for dynamic process management with an aspect-oriented framework, in: *Proceedings of the International Workshop on Dynamic Process Management (DPM 2006)*, Springer, Vienna, Austria. pp. 193–204.
- [19] Chiu, D.K., Li, Q., Karlapalem, K., 1999. Exception handling with workflow evolution in ADOME-WFMS: a taxonomy and resolution techniques. *ACM Siggroun Bulletin* 20, 8–8.
- [20] Comfort, L., Mosse, D., Znati, T., 2009. Managing risk in real time: Integrating information technology into disaster risk reduction and response. *Commonwealth: A Journal of Political Science* 15, 27–45.
- [21] Commission, I.E., 1990. *IEC 61025 Fault Tree Analysis (FTA)*.
- [22] Conforti, R., de Leoni, M., La Rosa, M., van der Aalst, W.M.P., 2013. Supporting Risk-Informed Decisions during Business Process Execution. *QUT ePrints* 55979. Queensland University of Technology. <http://eprints.qut.edu.au/55979>.
- [23] Conforti, R., Fortino, G., La Rosa, M., ter

- Hofstede, A.H.M., 2011. History-aware, real-time risk detection in business processes, in: Proc. of CoopIS, Springer.
- [24] Conforti, R., ter Hofstede, A.H.M., La Rosa, M., Adams, M., 2012. Automated risk mitigation in business processes, in: Proc. of CoopIS, Springer.
- [25] Cope, E.W., Küster, J.M., Etzweiler, D., 2009. Risk Extensions to the BPMN 1.1 Business Process Metamodel. Technical Report RZ3740. IBM Research.
- [26] Cope, E.W., Küster, J.M., Etzweiler, D., Deleris, L.A., Ray, B., 2010. Incorporating risk into business process models. IBM Journal of Research and Development 54, 4.
- [27] Cousins, P.D., Lamming, R.C., Bowen, F., 2004. The role of risk in environment-related supplier initiatives. International Journal of Operations & Production Management 24, 554–565.
- [28] Davis, F.D., 1989. Perceived usefulness, perceived ease of use, and user acceptance of information technology. MIS quarterly 13, 319–340.
- [29] Davis, R.B., Brabander, E., 2007. ARIS Design Platform: Getting Started with BPM. Springer.
- [30] Dayal, U., Buchmann, A.P., McCarthy, D.R., 1988. Rules are objects too: a knowledge model for an active, object-oriented database system, in: Advances in Object-Oriented Database Systems. Springer, pp. 129–143.
- [31] Dumas, M., van der Aalst, W.M., ter Hofstede, A.H., 2005. Process-Aware Information Systems: Bridging People and Software through Process Technology. Wiley & Sons.
- [32] Ermotti, S., 2011. Internal message from Sergio P. Ermotti, Group CEO, about the unauthorized trading incident. http://www.ubs.com/global/en/about_ubs/media/global/unauthorized_trading_incident/2011-10-05-internal-message-ermotti.html (Accessed 16 May 2013).
- [33] Faulds, F., Bessis, J., 2013. Rogue trading: Back to front. Journal of Risk Management in Financial Institutions 6, 4–5.
- [34] Fenz, S., 2010. From the resource to the business process risk level, in: Proceedings of the South African Information Security Multi-Conference (SAISMC’2010), pp. 100–109.
- [35] Fenz, S., Neubauer, T., 2009. How to determine threat probabilities using ontologies and bayesian networks, in: Proceedings of the 5th Annual Workshop on Cyber Security and Information Intelligence Research: Cyber Security and Information Intelligence Challenges and Strategies, ACM, New York, NY, USA. pp. 69:1–69:3.
- [36] Gay, P., Pla, A., López, B., Meléndez, J., Meunier, R., 2010. Service workflow monitoring through complex event processing, in: ETFA, IEEE. pp. 1–4.
- [37] Golden, W., Acton, T., Conboy, K., van der Heijden, H., Tuunainen, V.K. (Eds.), 2008. 16th European Conference on Information Systems, ECIS 2008, Galway, Ireland, 2008.
- [38] Günther, C.W., Verbeek, E., 2013. Supporting Risk-Informed Decisions during Business Process Execution. Technical Report 55979. Eindhoven University of Technology. http://www.xes-standard.org/_media/openxes/openxesdeveloperguide-1.9.pdf.
- [39] Halpin, T., Bloesch, A., 1999. Data modeling in uml and orm: a comparison. Journal of Database Management (JDM) 10, 4 – 13.
- [40] Harland, C., Brenchley, R., Walker, H., 2003. Risk in supply networks. Journal of Purchasing and Supply Management 9, 51 – 62.
- [41] Hermosillo, G., Seinturier, L., Duchien, L., 2010. Using complex event processing for dynamic business process adaptation, in: IEEE SCC, IEEE Computer Society. pp. 466–473.
- [42] Herrmann, P., Herrmann, G., 2006. Security requirement analysis of business processes. Electronic Commerce Research 6, 305–335.
- [43] Hespos, R.F., Strassmann, P.A., 1965. Stochastic decision trees for the analysis of investment decisions. Management Science 11, 244–259.
- [44] High Court of Australia, . Patel v The Queen [2012] HCA 29 (24 August 2012). <http://www.austlii.edu.au/au/cases/cth/HCA/2012/29.html> Accessed 2 May 2013.
- [45] Hollingsworth, D., 1995. The Workflow Reference Model. Workflow Management Coalition. Workflow Management Coalition.
- [46] Islam, M.M.Z., Bhuiyan, M., Krishna, A., Ghose, A., 2009. An integrated approach to managing business process risk using rich organizational models, in: Meersman, R., Dil-

- lon, T.S., Herrero, P. (Eds.), OTM Conferences (1), Springer. pp. 273–285.
- [47] Jacque, L., 2010. Global Derivative Debacles: From Theory to Malpractice. World Scientific. chapter 11: Société Générale. pp. 179–196.
- [48] Jakoubi, S., Goluch, G., Tjoa, S., Quirchmayr, G., 2008. Deriving resource requirements applying risk-aware business process modeling and simulation, in: [37]. pp. 1542–1554. pp. 1542–1554.
- [49] Jallow, A.K., Majeed, B., Vergidis, K., Tiwari, A., Roy, R., 2007. Operational risk analysis in business processes. *BT Technology Journal* 25, 168–177.
- [50] Johnson, W.G., 1973. MORT - The Management Oversight and Risk Tree. U.S. Atomic Energy Commission.
- [51] Kaegi, M., Mock, R., Ziegler, R., Nibali, R., 2006. Information systems’ risk analysis by agent-based modelling of business processes, in: Soares, C.G., Zio, E. (Eds.), *ESREL*, Taylor & Francis. pp. 2277–2284.
- [52] Kang, B., Cho, N.W., Kang, S.H., 2009. Real-time risk measurement for business activity monitoring (bam). *International Journal of Innovative Computing, Information and Control* 5, 3647–3657.
- [53] Karagiannis, D., Mylopoulos, J., Schwab, M., 2007. Business process-based regulation compliance: The case of the sarbanes-oxley act, in: *RE, IEEE*. pp. 315–321.
- [54] La Rosa, M., ter Hofstede, A.H.M., Wohed, P., Reijers, H.A., Mendling, J., van der Aalst, W.M.P., 2011a. Managing process model complexity via concrete syntax modifications. *Industrial Informatics, IEEE Transactions on* 7, 255–265.
- [55] La Rosa, M., Wohed, P., Mendling, J., ter Hofstede, A.H.M., Reijers, H.A., van der Aalst, W.M.P., 2011b. Managing process model complexity via abstract syntax modifications. *Industrial Informatics, IEEE Transactions on* 7, 614–629.
- [56] Lambert, J.H., Jennings, R.K., Joshi, N.N., 2006. Integration of risk identification with business process models. *Systems engineering* 9, 187–198.
- [57] Little, A., Best, P.J., 2003. A framework for separation of duties in an sap r/3 environment. *Managerial Auditing Journal* 18, 419–430.
- [58] Lund, M.S., Solhaug, B., Stølen, K., 2011. *Model-Driven Risk Analysis*. Springer.
- [59] Luo, Z., Sheth, A., Kochut, K., Arpinar, B., 2003. Exception handling for conflict resolution in cross-organisational workflows. *Distributed and Parallel Databases* 11, 271–306.
- [60] Mendling, J., Strembeck, M., Recker, J., 2012. Factors of process model comprehension - findings from a series of experiments. *Decision Support Systems* 53, 195–206.
- [61] Metropolis, N., 1987. The beginning of the monte carlo method. *Los Alamos Science* 15, 125–130.
- [62] Meulbroek, L., 2000. Total strategies for company-wide risk control. *Financial Times* 9, 1–4.
- [63] Meyer, B., 1990. *Introduction to the theory of programming languages*. Prentice-Hall.
- [64] Mock, R., Corvo, M., 2005. Risk analysis of information systems by event process chains. *International journal of critical infrastructures* 1, 247–257.
- [65] Moghim, F.H., Zadeh, H., Wickramasinghe, N., 2012. An intelligence e-risk detection model to improve decision efficiency in the context of the orthopaedic operating room, in: *Critical Issues for the Development of Sustainable E-health Solutions*. Springer, pp. 17–32.
- [66] Mollenkamp, C., Sonne, P., Hall, D., 2011. Rogue trading lasted 3 years. *The Wall Street Journal*.
- [67] Neiger, D., Churilov, L., zur Muehlen, M., Rosemann, M., 2006. Integrating risks in business process models with value focused process engineering, in: Ljungberg, J., Andersson, M. (Eds.), *ECIS*, pp. 1606–1615.
- [68] Object Management Group (OMG), 2011. *Business Process Model and Notation (BPMN) ver. 2.0*. Object Management Group (OMG). URL: <http://www.omg.org/spec/BPMN/2.0>.
- [69] Oracle, <http://download.oracle.com/docs/cd/E15523.01/integration.1111/e10224/bp-sensors.htm>. Accessed: June 2011. *BPEL Process Manager Developer’s Guide*.
- [70] Panayiotou, N.A., Oikonomitsios, S., Athanasiadou, C., Gayialis, S.P., . Risk assessment in virtual enterprise networks: A process-driven internal audit approach,

- in: *Managing Risk in Virtual Enterprise Networks: Implementing Supply Chain Principles*. IGI Global.
- [71] Recker, J., 2010a. Continued use of process modeling grammars: the impact of individual difference factors. *European Journal of Information Systems* 19, 76–92.
 - [72] Recker, J., 2010b. Explaining usage of process modeling grammars: Comparing three theoretical models in the study of two grammars. *Information & management* 47, 316–324.
 - [73] Recker, J., 2010c. Opportunities and constraints: the current struggle with bpmn. *Business Process Management Journal* 16, 181–201.
 - [74] Recker, J., La Rosa, M., 2012. Understanding user differences in open-source workflow management system usage intentions. *Information Systems* 37, 200–212.
 - [75] Rikhardsson, P., Best, P.J., Green, P., Rosemann, M., 2006. Business process risk management and internal control: A proposed research agenda in the context of compliance and ERP systems, in: *Proceedings of the Second Asia/Pacific Research Symposium on Accounting Information Systems*, Melbourne, Australia.
 - [76] Rosemann, M., zur Muehlen, M., 2005. Integrating risks in business process models, in: *ACIS, AISel*.
 - [77] Rotaru, K., Wilkin, C., Churilov, L., Neiger, D., 2008. Formalising risk with value-focused process engineering, in: [37]. pp. 1583–1595.
 - [78] Rotaru, K., Wilkin, C., Churilov, L., Neiger, D., Ceglowski, A., 2011. Formalizing process-based risk with value-focused process engineering. *Information Systems and e-Business Management* 9, 447–474.
 - [79] Salzer, E., 2011. Live aus der praxisfallbeispiele für spezifische kommunikationsstrategien, in: *Quintessenz der Unternehmenskommunikation*. Springer, pp. 46–90.
 - [80] Schmitt, M., Grégoire, B., Dubois, E., 2005. A risk based guide to business process design in inter-organizational business collaboration, in: *International Workshop on Requirements Engineering for Business Need and IT Alignment (REBNITA 2005)*.
 - [81] Schwartz, P., 2000. When good companies do bad things. *Strategy & Leadership* 28, 4–11.
 - [82] Sienou, A., Karduck, A.P., Lamine, E., Pingaud, H., 2008. Business process and risk models enrichment: Considerations for business intelligence, in: *ICEBE*, pp. 732–735.
 - [83] Sienou, A., Karduck, A.P., Pingaud, H., 2006. Towards a framework for integrating risk and business process management, in: Dolgui, A., Morel, G., Pereira, C.E. (Eds.), *Information Control Problems in Manufacturing*. Elsevier. volume 6, pp. 615–620.
 - [84] Sienou, A., Lamine, E., Pingaud, H., Karduck, A.P., 2009. Aspects of the bpm language for risk driven process engineering, in: Meersman, R., Herrero, P., Dillon, T.S. (Eds.), *OTM Workshops*, Springer. pp. 172–183.
 - [85] Sienou, A., Lamine, E., Pingaud, H., Karduck, A.P., 2010. Risk driven process engineering in digital ecosystems: Modelling risk, in: *DEST*, pp. 647–650.
 - [86] Simons, R.L., 1999. How risky is your company? *Harvard Business Review* 77, 85.
 - [87] Singh, P., Gelgi, F., Davulcu, H., Yau, S.S., Mukhopadhyay, S., 2008. A risk reduction framework for dynamic workflows, in: *IEEE SCC (1)*, IEEE Computer Society. pp. 381–388.
 - [88] Smallman, C., 1996. Risk and organizational behaviour: a research model. *Disaster Prevention and Management* 5, 12–26.
 - [89] Standards Australia and Standards New Zealand, 2009. Standard AS/NZS ISO 31000.
 - [90] Straumann, T., 2010. The UBS Crisis in Historical Perspective. Technical Report. Institute for Empirical Research in Economics, University of Zurich.
 - [91] Strecker, S., Heise, D., Frank, U., 2010. RiskM: A multi-perspective modeling method for IT risk assessment. *Information Systems Frontiers* 13, 1–17.
 - [92] Suriadi, S., Weiß, B., Winkelmann, A., ter Hofstede, A., Wynn, M., Ouyang, C., Adams, M., Conforti, R., Fidge, C., La Rosa, M., Pika, A., 2012. Current Research in Risk-Aware Business Process Management - Overview, Comparison, and Gap Analysis. BPM Center Report BPM-12-13. BPMcenter.org.
 - [93] Sybase, <http://www.sybase.com.au/files/>

- White_Papers/Sybase_CEP_Implementation_Methodology_wp.pdf. Accessed: June 2011. Sybase CEP Implementation Methodology for Continuous Intelligence.
- [94] Taylor, P., Godino, J.J., Majeed, B., 2008. Use of fuzzy reasoning in the simulation of risk events in business processes, in: Louca, L., Chrysanthou, Y., Oplatkova, Z., AlBe-gain, K. (Eds.), *Proceedings of the 22nd European Conference on Modelling and Simulation*, pp. 25–30.
 - [95] ter Hofstede, A.H.M., van der Aalst, W.M.P., Adams, M., Russell, N., 2010. *Modern Business Process Automation: YAWL and its Support Environment*. Springer.
 - [96] Thomas, H., 2010. *Sick to Death*. Allen & Unwin.
 - [97] Tjoa, S., Jakoubi, S., Quirchmayr, G., 2008. Enhancing business impact analysis and risk assessment applying a risk-aware business process modeling and simulation methodology, in: *ARES*, IEEE Computer Society. pp. 179–186.
 - [98] Voluntary Interindustry Commerce Solutions Association, <http://www.vics.org>. Accessed: June 2011. Voluntary Inter-industry Commerce Standard (VICS).
 - [99] Wang, D., Rundensteiner, E.A., Ellison, R.T., Wang, H., 2011. Active complex event processing infrastructure: Monitoring and reacting to event streams, in: Abiteboul, S., Böhm, K., Koch, C., Tan, K.L. (Eds.), *ICDE Workshops*, IEEE. pp. 249–254.
 - [100] Weiß, B., Winkelmann, A., 2011. Developing a process-oriented notation for modeling operational risks - a conceptual metamodel approach to operational risk management in knowledge intensive business processes within the financial industry, in: *HICSS*, IEEE Computer Society. pp. 1–10.
 - [101] Woo, G., 2008. *Terrorism Risk*. Wiley Handbook of Science and Technology for Homeland Security. 2:1–17. John Wiley & Sons.
 - [102] zur Muehlen, M., Ho, D.T.Y., 2005. Risk management in the bpm lifecycle, in: Bus-sler, C., Haller, A. (Eds.), *Business Process Management Workshops*, pp. 454–466.

Appendix A. Detailed Abstract Syntax

This Appendix provides a detailed description of each element of the abstract syntax of the sensor definition language.

Definition	Description
$Sensor \triangleq v : Variables; t : Trigger; riskCon : RiskCon; faultCon : BoolCon; consequence : MaCon;$	A sensor is composed of a set of variables, a trigger, a risk condition, a fault condition, and a consequence.
$Trigger \triangleq timer \mid event$	A trigger can either be timer or event based.
$Variables \triangleq Assignment^+$	A set of variables is a several assignments.
$Assignment \triangleq result : varName; i : Info$	A variable assigns a name to an information.
$Info \triangleq VarFun \mid Definition \mid CaseExp \mid CaseEleExp$	An information is a function invoked on a variable, a definition, or a piece of information collected from the process instance (i.e. <i>CaseExp</i> or <i>CaseEleExp</i>).
$VarFun \triangleq ResCon \mid ResSimFun \mid ResComFun$	The three types of functions invoked on variables are available: <i>ResCon</i> , <i>ResSimFun</i> or <i>ResComFun</i> .
$Definition \triangleq c : constant$	A definition is a constant.
$CaseExp \triangleq cis : CaseIDStat; a : Action$	A piece of information related to the process instance.
$CaseEleExp \triangleq ce : CaseExp; ton : TaskOrNet$	A piece of information related to an element (i.e. a task or a net) of the process instance.
$TaskOrNet \triangleq taskLabel \mid netName$	The identifier of a task or of a net.
$CaseIDStat \triangleq absExp \mid relExp \mid CaseConSet$	Identifies an instance or a set of instances.
$Action \triangleq predAct \mid taskOrNetVar \mid SubVarExp \mid inputPredAct$	An action is either a predefined action, a task variable, a net variable, a subvariable, or a predefined action that requires an input.
$SubVarExp \triangleq var^+$	A subvariable of a task variable or of a net variable.
$CaseConSet \triangleq CaseCon \mid CaseParam \mid CaseConExp$	The three types of identifiers for a set of instances.
$CaseCon \triangleq ton : TaskOrNet; a : Action; co : CompOp; rhe : RHExp$	Identifies instances for which a specific piece of information is compared with a <i>RHExp</i> .
$CaseParam \triangleq i : idFun; co : CompOp; rhe : RHExp$	Identifies instances for which the instance ID is compared with a <i>RHExp</i> .
$CaseConExp \triangleq ccs_1, ccs_2 : CaseConSet; bo : BoolOp$	Identifies instances for which several conditions are satisfied.
$CompOp \triangleq l \mid leq \mid g \mid geq \mid eq \mid contains \mid isContained$	Set of comparison operators.
$RHExp \triangleq constant \mid function \mid varName \mid RHExpSet$	Can be a constant, a function, a function invoked on a variable, or a <i>RHExpSet</i> .
$RHExpSet \triangleq rhe_1, rhe_2 : RHExp; o : Op$	An arithmetical expression built using <i>RHExp</i> .
$RiskCon \triangleq riskLikelihood, riskThreshold : MaCon$	A risk condition is composed of two <i>MaCon</i> .
$MaCon \triangleq MaITE \mid MaExp \mid MaFor \mid FixMaFor \mid ResSimFun \mid ResComFun \mid varName \mid constant$	A arithmetical condition can be an if-then-else expression, an expression, a loop, a function on a variable, a variable name, or a constant.
$MaITE \triangleq if : BoolCon; then, else : MaCon$	A arithmetical condition that varies based on the result of a boolean condition.
$MaExp \triangleq MaUnExp \mid MaBinExp$	The two types of arithmetical expression: <i>MaUnExp</i> or <i>MaBinExp</i> .
$MaUnExp \triangleq s : sub; me : MaCon$	A negative arithmetical expression.
$MaBinExp \triangleq me_1, me_2 : MaCon; mo : MaOp$	A arithmetical operation between <i>MaCons</i> .
$MaOp \triangleq add \mid sub \mid mul \mid div \mid exp \mid mod$	The set of arithmetical operators.
$BoolCon \triangleq BoolITE \mid BoolExp \mid BoolFor \mid FixBoolFor \mid Comp \mid ResCon \mid varName \mid constant$	A boolean condition can be an if-then-else expression, an expression, a loop, a comparison, a function on a variable, a variable, or a constant.
$BoolITE \triangleq if, then, else : BoolExp$	A boolean condition that varies based on the result of another boolean condition.
$BoolExp \triangleq BoolUnExp \mid BoolBinExp$	The two types of boolean expression: <i>BoolUnExp</i> or <i>BoolBinExp</i> .
$BoolUnExp \triangleq n : neg; e : BoolCon$	A negated boolean expression.
$BoolBinExp \triangleq e_1, e_2 : BoolCon; bo : BoolOp$	A boolean operation between <i>BoolCons</i> .
$BoolOp \triangleq and \mid or$	The set of boolean operators.

Definition	Description
$Comp \triangleq ce_1, ce_2 : CompElem; co : CompOp$	A comparison between two <i>CompElems</i> .
$CompElem \triangleq MaCon \mid ResListFun \mid varName \mid constant$	A comparison element can be a arithmetical condition, a function invoked on a variable, a variable, or a constant.
$ResCon \triangleq res : varName; a : Activity$	A function invoked on a variable using an activity.
$ResListFun \triangleq result : varName; lrf : listResFun$	A function invoked on a variable. It returns a list.
$ResSimFun \triangleq resource : varName; srf : simResFun$	A function invoked on a variable which is a resource.
$ResComFun \triangleq res_1, res_2 : varName; crf : comResFun$	A function invoked on two variables which are resources.
$Activity \triangleq resource : varName; lrf : ResListFun$	A function invoked on a variable which is a resources.
$MaFor \triangleq t : TypeMF; lr : ListResult; fme : ForMaCon$	A nested loop that cycles on each element of a <i>ListResult</i> and aggregates the result of a <i>ForMaCon</i> based on a <i>TypeMF</i> .
$FixMaFor \triangleq fixEle : varName; mf : MaFor$	A <i>MaFor</i> that cycles on each element of a <i>ListResult</i> using as index of the loop the element <i>fixEle</i> .
$TypeMF \triangleq add \mid mul$	Two forms of aggregations: adding and multiplying.
$BoolFor \triangleq t : TypeBF; lr : ListResult; fbe : ForBoolCon$	A nested loop that cycles on each element of a <i>ListResult</i> and aggregates the result of a <i>ForBoolCon</i> based on a <i>TypeBF</i> .
$FixBoolFor \triangleq fixEle : varName; bf : BoolFor$	A <i>BoolFor</i> that cycles on each element of a <i>ListResult</i> using as index of the loop the element <i>fixEle</i> .
$TypeBF \triangleq and \mid or$	Two forms of aggregations: AND or OR.
$ListResult \triangleq varName^+$	A list of variables.
$ForMaCon \triangleq ForMaITE \mid ForMaExp \mid constant \mid varName$	A arithmetical condition can be an if-then-else expression, an expression, a constant, or a variable.
$ForMaITE \triangleq if : ForBoolCon; then, else : ForMaCon$	A arithmetical condition that varies based on the result of a boolean condition.
$ForMaExp \triangleq ForMaUnExp \mid ForMaBinExp$	The two types of arithmetical expression: <i>ForMaUnExp</i> or <i>ForMaBinExp</i> .
$ForMaUnExp \triangleq s : sub; me : ForMaCon$	A negative arithmetical expression.
$ForMaBinExp \triangleq me_1, me_2 : ForMaCon; mo : MaOp$	A arithmetical operation between <i>ForMaCon</i> .
$ForBoolCon \triangleq ForBoolITE \mid ForBoolExp \mid ForComp \mid varName \mid constant$	A boolean condition can be an if-then-else expression, an expression, a comparison, a variable, or a constant.
$ForBoolITE \triangleq if, then, else : ForBoolExp$	A boolean condition that varies based on the result of another boolean condition.
$ForBoolExp \triangleq ForBoolUnExp \mid ForBoolBinExp$	The two types of boolean expression: <i>ForBoolUnExp</i> or <i>ForBoolBinExp</i> .
$ForBoolUnExp \triangleq n : neg; e : ForBoolCon$	A negated boolean expression.
$ForBoolBinExp \triangleq e_1, e_2 : ForBoolCon; bo : BoolOp$	A boolean operation between <i>ForBoolCons</i> .
$ForComp \triangleq ce_1, ce_2 : ForCompElem; co : CompOp$	A comparison between two <i>ForCompElems</i> .
$ForCompElem \triangleq ForMaCon \mid varName \mid constant$	A comparison element can be a arithmetical condition, a variable, or a constant.

Appendix B. Actions

This appendix describes all actions available in the sensor definition language.

Action	Description
(ID)	returns the ID of the generic instance that is being analyzed
[IDCurr]	returns the ID of the instance that the sensor is monitoring
Count	returns the number of times a task has been completed
offerResource	returns the resources to which the task has been offered
allocateResource	returns the resources to which the task has been allocated
startResource	returns the resources that started the task
completeResource	returns the resource that completed the task
isOfferd	returns “true” if the task has been offered
isAllocated	returns “true” if the task has been allocated
isStarted	returns “true” if the task has been started
isCompleted	returns “true” if the task has been completed
OfferTime	returns the time when the task has been offered
AllocateTime	returns the time when the task has been allocated
StartTime	returns the time when the task has been started
CompleteTime	returns the time when the task has been completed
OfferTimeInMillis	returns the time (in millisecond) when the task has been offered
AllocateTimeInMillis	returns the time (in millisecond) when the task has been allocated
StartTimeInMillis	returns the time (in millisecond) when the task has been started
CompleteTimeInMillis	returns the time (in millisecond) when the task has been completed
PassTimeInMillis	returns the amount of time (in millisecond) that was needed to complete the task
TimeEstimationInMillis	returns an estimation of the time (in millisecond) needed to completed the task/process
Variable	returns the value of the variable or sub-variable required
offerDistribution	returns the list of resources to which the task is offered by default
allocateDistribution	returns the list of resources to which the task is allocated by default
startDistribution	returns the list of resources to which the task is started by default
offerInitiator	returns the offering policy of the task (user or system)
allocateInitiator	returns the allocating policy of the task (user or system)
startInitiator	returns the starting policy of the task (user or system)
CountElements	returns the number of instances that satisfy the parameters required
FraudProbabilityFunc	returns the probability of a fraud using as parameters: the current number of executions, the maximum number of executions allowed, the parameter used to group these instances, the parameter used to identify a temporal window, and the dimension of the temporal window

List of actions of the architecture

Appendix C. Nested Loops

This appendix describes the four types of nested loops available in the sensor definition language.

For	Description
forAND[] []	executes a nested loop for each list provided in input (among the first couple of brackets) resolving the expression (defined among the second couple of brackets), then returns the AND conjunction of the results obtained
forOR[] []	executes a nested loop for each list provided in input (among the first couple of brackets) resolving the expression (defined among the second couple of brackets), then returns the OR conjunction of the results obtained
forADD[] []	executes a nested loop for each list provided in input (among the first couple of brackets) resolving the expression (defined among the second couple of brackets), then returns the sum of the results obtained
forMUL[] []	executes a nested loop for each list provided in input (among the first couple of brackets) resolving the expression (defined among the second couple of brackets), then returns the product of the results obtained

List of nested loops

Appendix D. Functions

This appendix describes the functions available in the sensor definition language.

Function	Description
offeredList	returns the list of tasks currently offered to the resource/resources (returns a list of list if used on resources)
allocatedList	returns the list of task currently allocated to the resource/resources (returns a list of list if used on resources)
startedList	returns the list of task currently started by the resource/resources (returns a list of list if used on resources)
offeredNumber	returns the number of tasks currently offered to the resource/resources (returns a list if used on resources)
allocatedNumber	returns the number of tasks currently allocated to the resource/resources (returns a list if used on resources)
startedNumber	returns the number of tasks currently started by the resource/resources (returns a list if used on resources)
offeredMinNumber	returns the minimum number of tasks currently offered to the resource/resources
allocatedMinNumber	returns the minimum number of tasks currently allocated to the resource/resources
startedMinNumber	returns the minimum number of tasks currently started by the resource/resources
offeredMinNumberExcept	returns the minimum number of tasks currently offered to the resource/resources excluding the resource/resources provided in input (the input is provided using a dotted format after the name of the function)
allocatedMinNumberExcept	returns the minimum number of tasks currently allocated to the resource/resources excluding the resource/resources provided in input (the input is provided using a dotted format after the name of the function)
startedMinNumberExcept	returns the minimum number of tasks currently started by the resource/resources excluding the resource/resources provided in input (the input is provided using a dotted format after the name of the function)
offeredContain	returns <i>true</i> if the task provided in input (using a dotted format after the name of the function) is currently offered to the resource/resources.
allocatedContain	returns <i>true</i> if the task provided in input (using a dotted format after the name of the function) is currently allocated to the resource/resources.
startedContain	returns <i>true</i> if the resource/resources started the task provided in input (using a dotted format after the name of the function).

List of functions

Appendix E. Questionnaire

Part 1) Background Questions

E1a: Which description matches best your current status?

- Student
- Academic
- Professional

E1b: Please specify your gender:

- Female
- Male
- Prefer not to tell.

E2: How many years ago did you start process modeling?

_____ Years

E3a: How many process models have you analyzed or read within the last 12 months? (A year has about 250 work days. In case you read one model per day, this would sum up to 250 models per year)

_____ Models

E3b: How many process models have you created or edited within the last 12 months?

_____ Models

E3c: How many activities did all these models have on average?

_____ Activities

E4a: How many work days of formal training on process modeling have you received within the last 12 months? (This includes e.g. university lectures, certification courses, training courses. 10 weeks of a 120 minute university lecture is roughly 3 work days)

_____ Days

E4b: How many work days of self-education have you made within the last 12 months? (This includes e.g. learning-by-doing, self-study of textbooks or specifications)

_____ Days

Part 2) Your experience with Workflow Management Systems

Q1: Which of the following (process) modelling techniques other than YAWL have you used to describe a process or procedure? Tick all that apply.

- None. Please go to question 10.
- BPMN
- UML
- Activity Diagrams
- EPCs
- BPEL
- Petri Nets
- Protos
- Other:

Q2: If you have used any technique other than YAWL, roughly, how many **conceptual process** models do you think you have created?

----- process models

Q3: If you have used any technique other than YAWL, roughly, how many **workflow** models (i.e. executable process models) do you think you have read?

----- workflow models

Q4: How long have you been using a Workflow Management System?

- I am evaluating to do so/I have just started
- Less than 1 month
- 1 - 6 months
- 7 - 12 months
- More than 1 year

Q6: Indicate your level of agreement to the following statements on the given scale by circling the number that best describes your view on the statement.

	Strongly disagree	Disagree	Somewhat disagree	Normal	Somewhat agree	Agree	Strongly agree
Overall, I am very familiar with Work flow Management Systems.	1	2	3	4	5	6	7
I feel very confident in my understanding of Workflow Management Systems.	1	2	3	4	5	6	7
I feel very competent in using Workflow Management Systems.	1	2	3	4	5	6	7

Part3) Your experience with the YAWL system

Q1: How long have you been using YAWL?

- I am evaluating to do so/I have just started
- Less than 1 month
- 1 - 6 months
- 7 - 12 months
- More than 1 year

Q2: Please indicate, roughly, the typical extent of usage of YAWL. This includes any activity related to the YAWL system, e.g. development, reading documentation, modelling, executing or simulating YAWL workflows, and using a system that is based on some YAWL component. Keep in mind, a regular working days has eight hours (480 minutes).

- Not applicable
- On average, I spend ----- hours and ----- minutes on YAWL every working day.

Q3: Roughly, how many YAWL models do you think you have created or read?

----- YAWL models

Q4: Which features of the YAWL system have you ever used? Tick all that apply

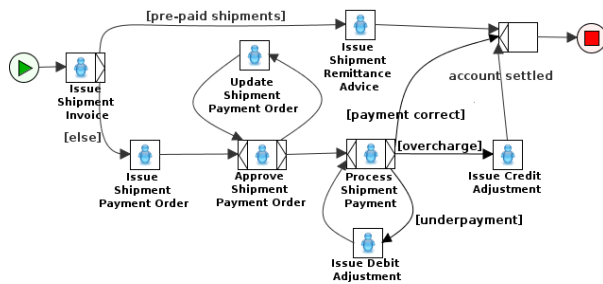
- Not applicable
- Execution environment

- Syntax checker/verification
- Cancellation region
- OR-join
- Multiple instance task
- Deferred choice
- Milestone
- Retain familiar/Separation of duties
- Deferred distribution
- Distribution set filter
- Allocation strategies
- Worklets/Exlets
- Custom services
- Configuration

Part 4) Risk Sensor Comprehension

In this part, you will be shown 3 risks related to a process described in YAWL. You will be asked questions about each of them.

Risk 1: Consider the following YAWL model and associated risk condition described below.



Variables:

A = case(current).Update Shipment Payment Order(Count)
 B = case(Update Shipment Payment Order(Count) $\geq E$).Update Shipment Payment Order(CountElements)
 C = case(Update Shipment Payment Order(Count) $\geq A$ AND Process Shipment Payment(isOffered)="true").Update Shipment Payment Order(CountElements)
 D = 0.6
 E = 5

Sensor Condition: $(C/B) > D$ where "/" is the division operator

Q0. How difficult is it to understand the meaning of the above sensor condition:

- 1 – very simple
- 2 – rather simple
- 3 – neutral
- 4 – rather difficult
- 5 – very difficult

Q1. Does the sensor send a notification if the task Update Shipment Payment Order may not be executed?

- Yes / No / I do not know

Q2. Can the sensor notify a risk if the task Update Shipment Payment Order has been executed only twice?

- Yes / No / I do not know

Q3. Does A retrieve the value of a variable named Count of the task Update Shipment Payment Order?

- Yes / No / I do not know

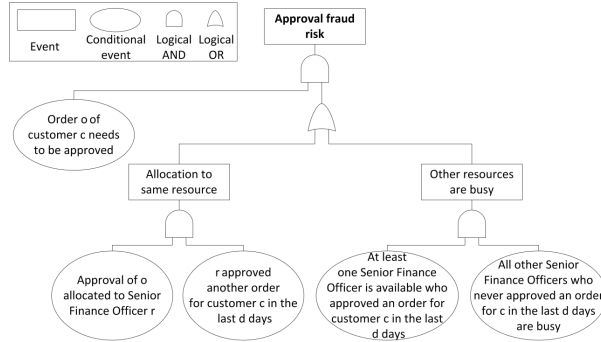
Q4. Does C return the number of instances where the task Process Shipment Payment has been offered?

- Yes / No / I do not know

Q5. Does D indicate a threshold that if exceeded produces a notification from the sensor?

- Yes / No / I do not know

Risk 2: Using the YAWL model of page 6 above, an Approval fraud risk has been identified using Fault Tree Analysis, as shown below.



To detect the risk of this fault, we first have to check whether there is an order, say order o of customer c, to be approved. This means checking that an instance of task Approve Shipment Payment Order is being executed. Moreover, we need to check that either of the following conditions holds:

1. o has been allocated to a Senior Finance Officer who has already approved another order for the same customer in the last d days; or
2. at least one Senior Finance Officer is available who approved an order for customer c in the last d days and all other Senior Finance Officers who never approved an order for c during the last d days are available

Q0. How difficult is it to understand the meaning of the above risk:

- 1 – very simple
- 2 – rather simple
- 3 – neutral
- 4 – rather difficult
- 5 – very difficult

Q1. Could condition A be used as trigger for the sensor?

- Yes / No / I do not know

Q2. If condition B is expressed using a variable, would the following assignment be correct?

B = case(current).Approve Shipment Payment Order(allocateResource)

- Yes / No / I do not know

Q3. Could condition D be expressed using only one variable in the risk condition?

- Yes / No / I do not know

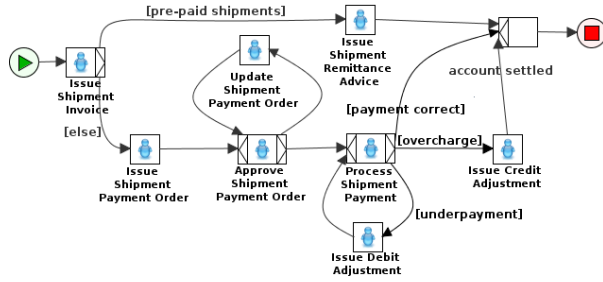
Q4. Could condition E be expressed using only one variable in the risk condition?

- Yes / No / I do not know

Q5. Could the AND between conditions B and C be expressed using only one variable in the risk condition?

- Yes / No / I do not know

Risk 3: Consider the following YAWL model and the associated risk condition described below.



Variables:

- A = case(current).Process Shipment Payment.Customer
- B = case(Process Shipment Payment.Customer=A AND Issue Debit Adjustment(isCompleted)="true"). Issue Debit Adjustment (CountElements)
- C = case(Process Shipment Payment.Customer=A AND Issue Credit Adjustment(isCompleted)="true"). Issue Debit Adjustment (CountElements)
- D = case(Process Shipment Payment.Customer=A).Process Shipment Payment(CountElements)
- F = 0.4

Sensor Condition:

$(B/D) > F | (C/D) > F$ where "/" is the division operator and "|" is the logical OR operator

Q0. How difficult is it to understand the meaning of the above sensor condition:

- 1 – very simple
- 2 – rather simple
- 3 – neutral
- 4 – rather difficult
- 5 – very difficult

Q1. Does the sensor send a notification if the task Issue Debit Adjustment is not executed?

- Yes / No / I do not know

Q2. Will the sensor send a notification as soon as the task Issue Credit Adjustment is completed?

- Yes / No / I do not know

Q3. Does B return the number of instances where the customer is the same of the current instance and the task Issue Debit Adjustment has been completed?

- Yes / No / I do not know

Q4. Does C return the number of instances where the task Process Shipment Payment has been offered?

- Yes / No / I do not know

Q5. Does B always return a value greater than the value return by C?

- Yes / No / I do not know

Part 5) Your views on the use of the Sensor-based component of the YAWL system

This part of the survey captures some information about how you overall rate the Sensor-based component of the YAWL system you have been using. Please note again that all information you provide will be treated confidentially. Thus, we ask you to please answer honestly.

In the following, you will be given a number of statements on opinions that you may have towards the Sensor-based component of the YAWL system. Please indicate your level of agreement to the

statements on the given scale by ticking the box that best describes your view on the respective statement.

Facilitating Conditions	Strongly disagree	Disagree	Somewhat disagree	Normal	Somewhat agree	Agree	Strongly agree
Guidance was available to me in the use of the Sensor-based component of the YAWL system.	1	2	3	4	5	6	7
Specialized instruction concerning the use of the Sensor-based component of the YAWL system was available to me.	1	2	3	4	5	6	7
A specific person or group was available for assistance with difficulties with the Sensor-based component of the YAWL system.	1	2	3	4	5	6	7
Satisfaction	Strongly disagree	Disagree	Somewhat disagree	Normal	Somewhat agree	Agree	Strongly agree
I am extremely pleased with my use of the Sensor-base component of the YAWL system.	1	2	3	4	5	6	7
I am extremely contented with my use of the Sensor-base component of the YAWL system.	1	2	3	4	5	6	7
I am extremely delighted with my use of the Sensor-base component of the YAWL system.	1	2	3	4	5	6	7
I am extremely satisfied with my use of the Sensor-base component of the YAWL system.	1	2	3	4	5	6	7

Perceived Usefulness	Strongly disagree	Disagree	Somewhat disagree	Normal	Somewhat agree	Agree	Strongly agree
Overall, I find the Sensor- based component of the YAWL system useful for modelling process-related risks.	1	2	3	4	5	6	7
I find the Sensor-based component of the YAWL system useful for achieving the purpose of modelling process-related risks.	1	2	3	4	5	6	7
I find the Sensor-based component of the YAWL system helps me in meeting my process-related risks modelling objectives.	1	2	3	4	5	6	7
Perceived Ease of Use	Strongly disagree	Disagree	Somewhat disagree	Normal	Somewhat agree	Agree	Strongly agree
I find it easy to model process-related risks in the way I intended using the Sensor-based component of the YAWL system.	1	2	3	4	5	6	7
I find learning to use the Sensor-based component of the YAWL system is easy.	1	2	3	4	5	6	7
I find easy to create process-related risks using the Sensor-based component of the YAWL system.	1	2	3	4	5	6	7

Intention to Use	Strongly disagree	Disagree	Somewhat disagree	Normal	Somewhat agree	Agree	Strongly agree
I intend to use the Sensor-based component of the YAWL system when I have to define and detect risks in business processes.	1	2	3	4	5	6	7
I predict I would use the Sensor-based component of the YAWL system.	1	2	3	4	5	6	7
I plan to use the Sensor-based component of the YAWL system in the future.	1	2	3	4	5	6	7
I prefer to continue to work with the Sensor-based component of the YAWL system.	1	2	3	4	5	6	7